

Unidad 5

Boosting. Definición, pasos. Comparación con bagging. Boosting para clasificación y para regresión. Adaboost. Decision stumps. Algoritmo de adaboost. Gradient Boosting Machines. Pasos. Algoritmo GBM. XGBoost. Pasos. Algoritmo XGBoost. Otras variables de boosting (LightGBM, Catboost). Aplicaciones Espaciales.

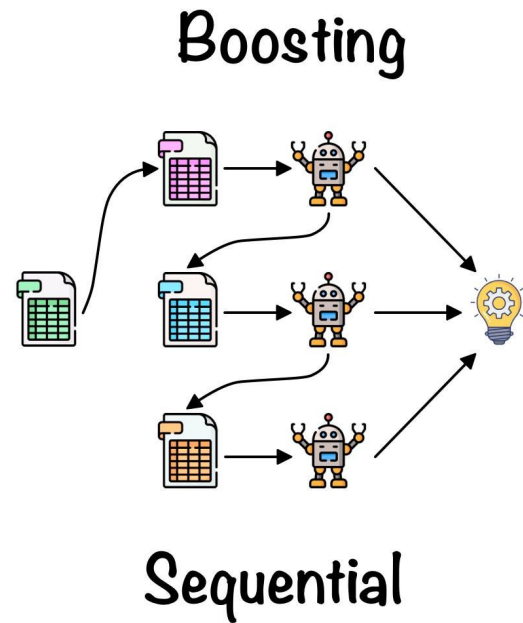
Generalidades

El **Boosting** es un **meta-algoritmo** de **aprendizaje automático** que se utiliza para mejorar la precisión de los modelos. Es una técnica de **aprendizaje conjunto (ensemble learning)**.

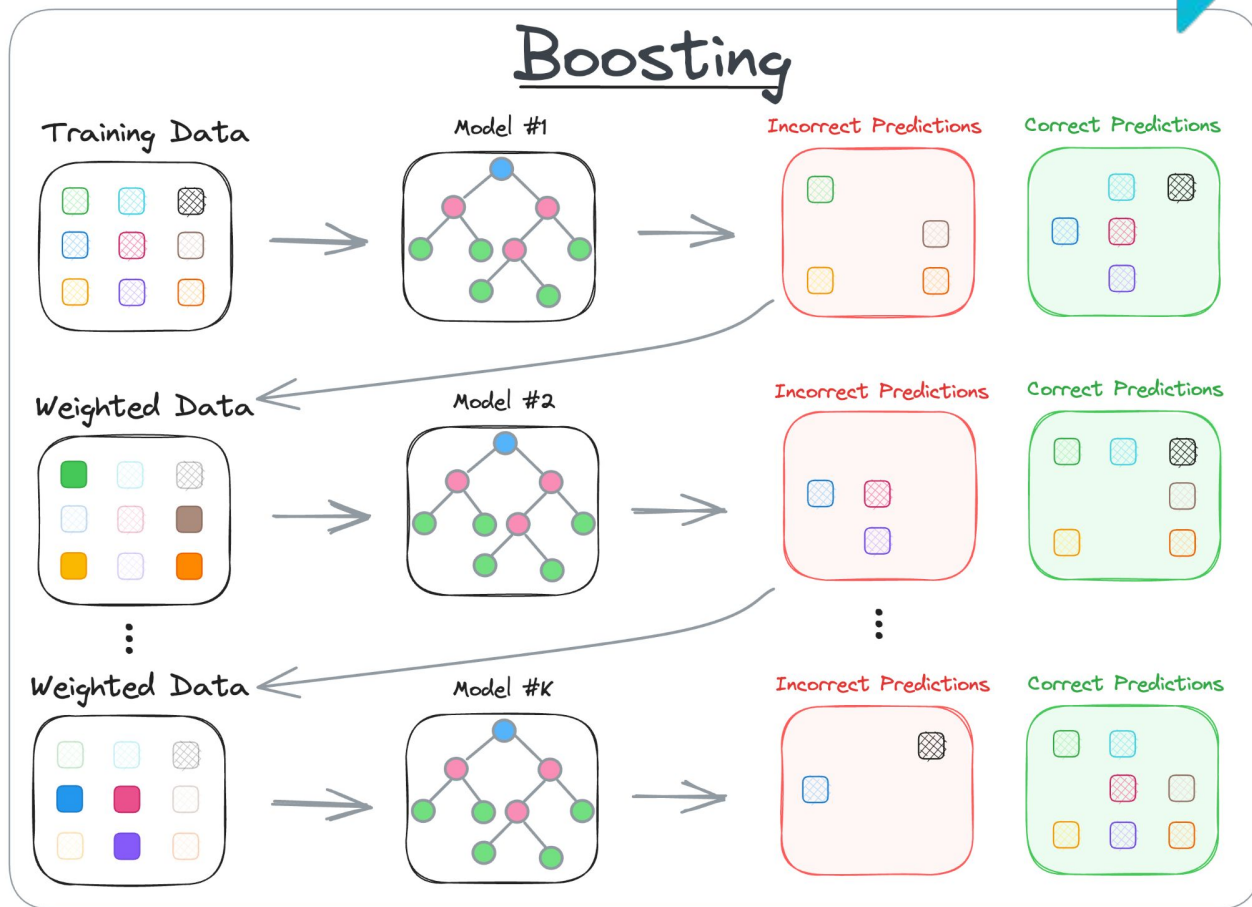
El boosting combina **múltiples modelos de aprendizaje "débiles"** (o *weak learners*), que individualmente apenas superan el rendimiento de una elección al azar, de manera **secuencial y adaptativa** para crear un **modelo "fuerte"** (*strong learner*) con alta capacidad predictiva.

Pasos del boosting

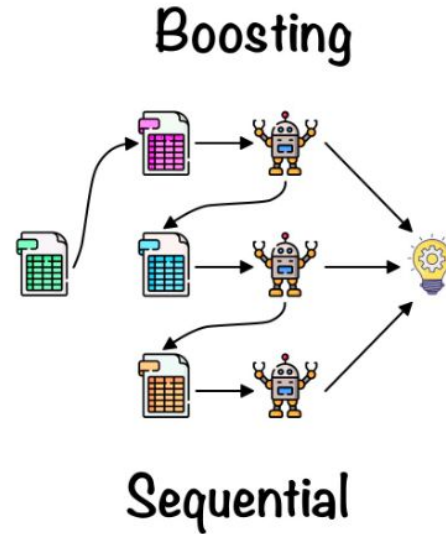
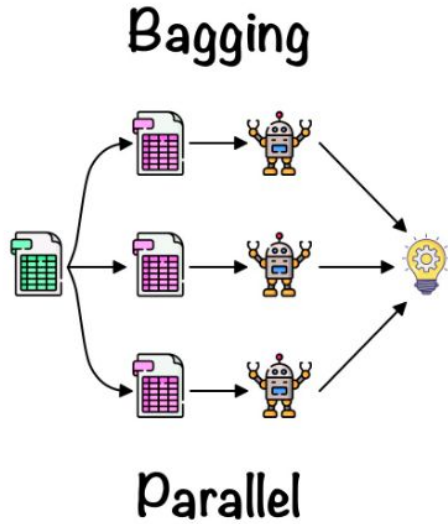
1. Se entrena un **primer modelo débil** sobre el conjunto de datos.
2. En cada iteración **secuencial**, se entrena un **nuevo modelo débil**. Este nuevo modelo se enfoca en **corregir los errores** cometidos por los modelos anteriores, prestando **mayor atención a las instancias (datos) que fueron clasificadas incorrectamente**.
3. Los modelos débiles se combinan para formar el clasificador final. La contribución de cada modelo se pondera, generalmente dando **mayor peso a aquellos que tuvieron un mejor rendimiento**.



Boosting

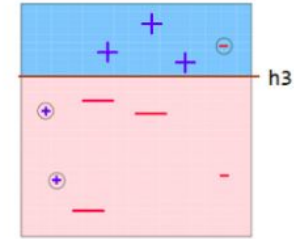
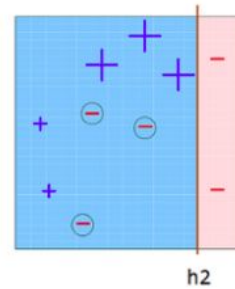
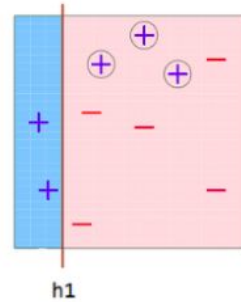
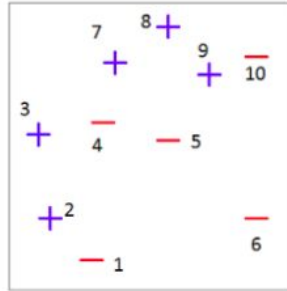


Comparación con bagging



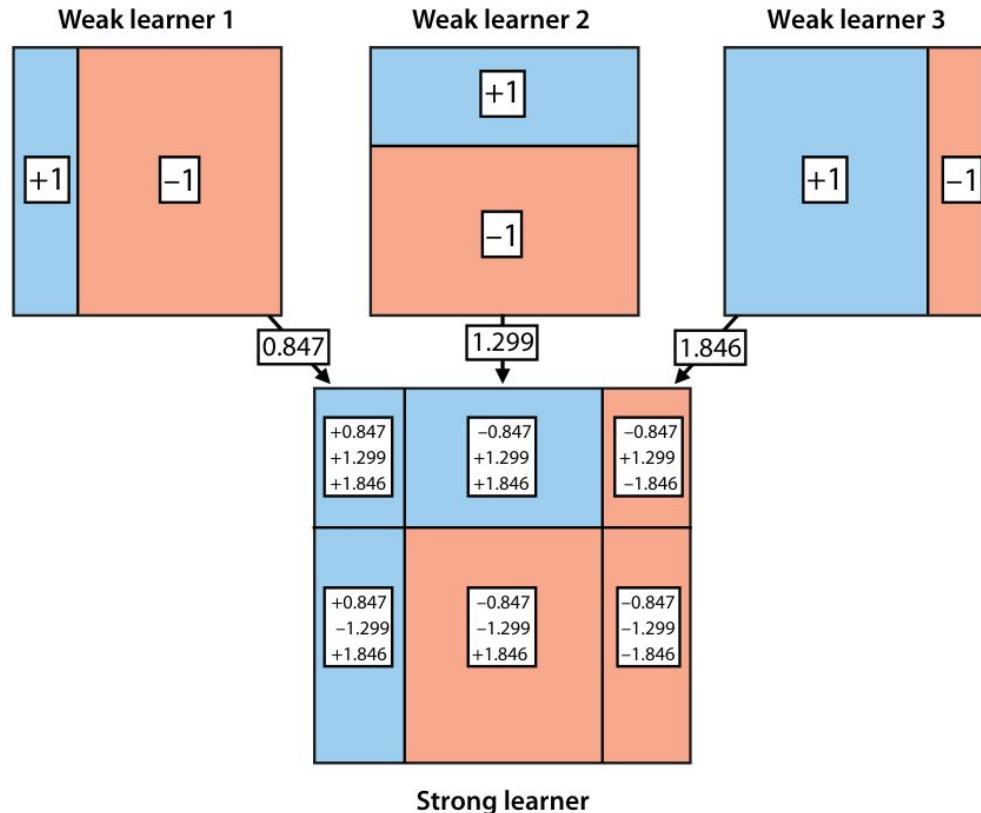
Recordemos que en **bagging** vamos a construir modelos de manera **independiente**, y luego hacer un **promedio** de esas predicciones. La combinación de modelos suele ser **más potente** que un algoritmo que el estimador base por separado.

Boosting para clasificación



$$H_{\text{final}} = \text{sign} \left(\alpha_1 \begin{array}{|c|} \hline \text{blue} \\ \hline \text{pink} \end{array} + \alpha_2 \begin{array}{|c|} \hline \text{blue} \\ \hline \text{pink} \end{array} + \alpha_3 \begin{array}{|c|} \hline \text{blue} \\ \hline \text{pink} \end{array} \right) = \begin{array}{|c|} \hline \text{blue} \\ \hline \text{pink} \end{array}$$

Boosting para clasificación



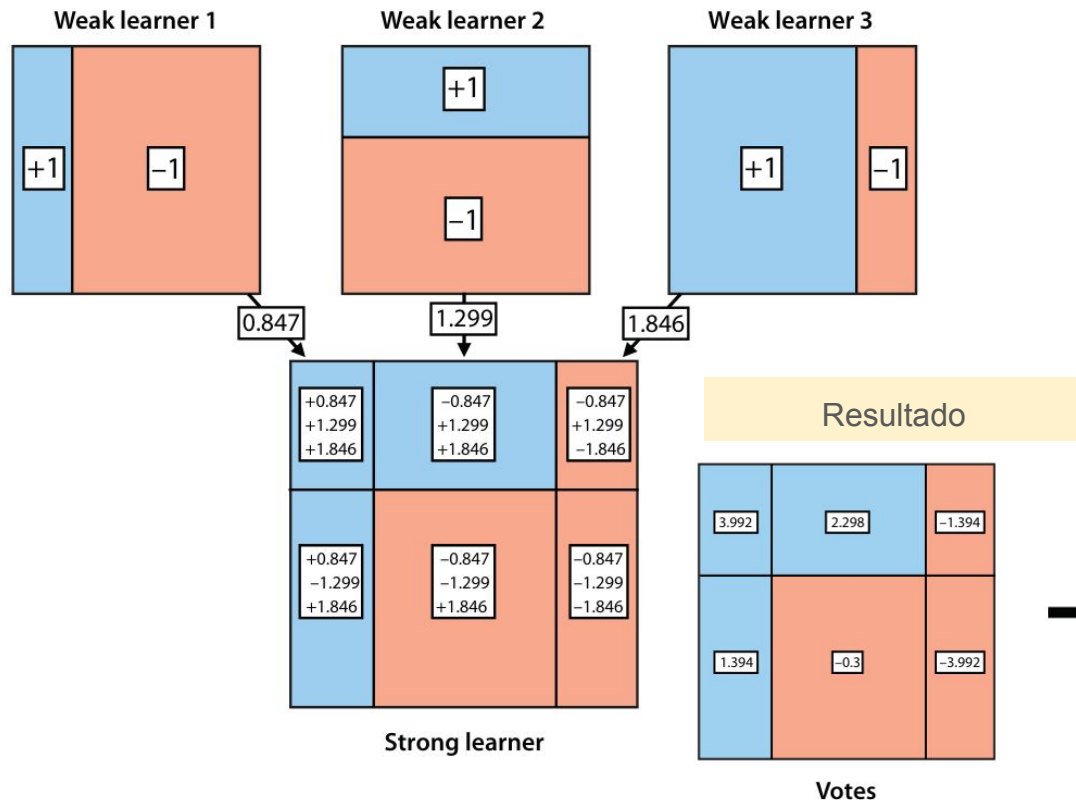
Aprendizaje Iterativo: Cada "Weak Learner" se entrena secuencialmente, corrigiendo los errores del clasificador anterior.

Ponderación de Errores: En cada paso, se asigna un **mayor peso** a los ejemplos que fueron **mal clasificados** por el clasificador anterior.

Contribución (alpha): Cada clasificador débil tiene una **ponderación de voto** específica (los valores 0.847, 1.299, 1.846). Este valor alpha es mayor cuanto menor sea el error del clasificador.

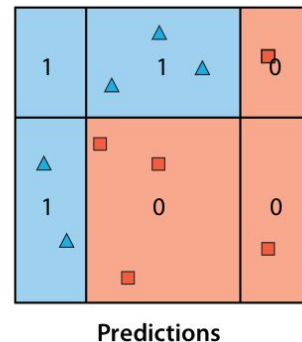
Strong Learner (Clasificador Final): La decisión final es la **suma ponderada** de los votos de todos los clasificadores débiles.

Adaboost: énfasis en los errores



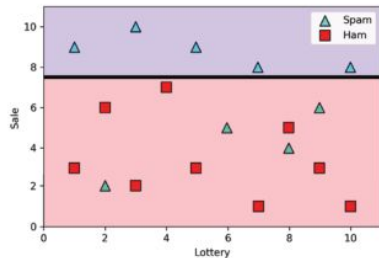
Votos (Scores): La zona de "Votes" muestra la suma de las ponderaciones de los clasificadores débiles para cada región.

Predicciones: El clasificador fuerte decide la clase (azul o rojo) basándose en el **signo** del voto acumulado (positivo = azul, negativo = rojo).

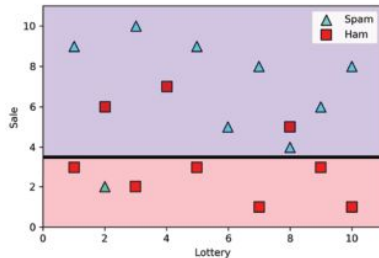


Boosting para clasificación

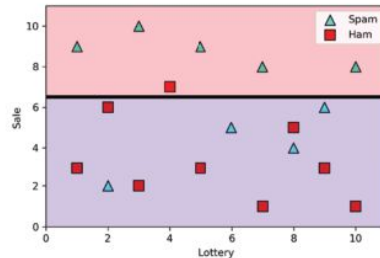
Weak learner 1



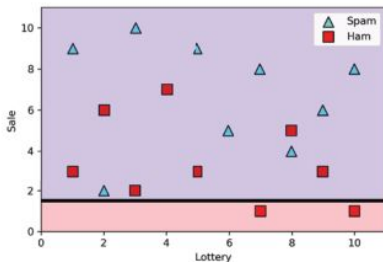
Weak learner 2



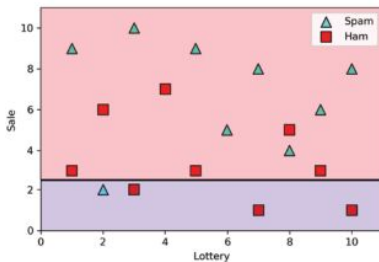
Weak learner 3



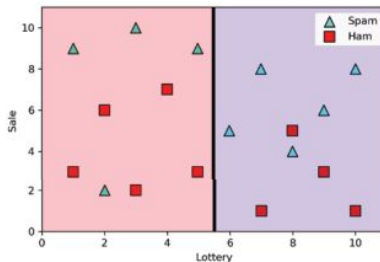
Weak learner 4



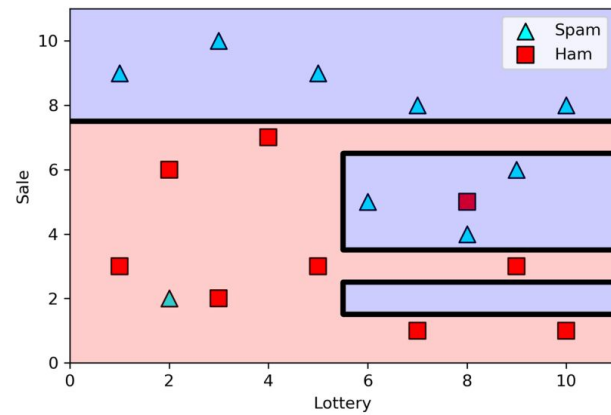
Weak learner 5



Weak learner 6



Resultado



Boosting para clasificación

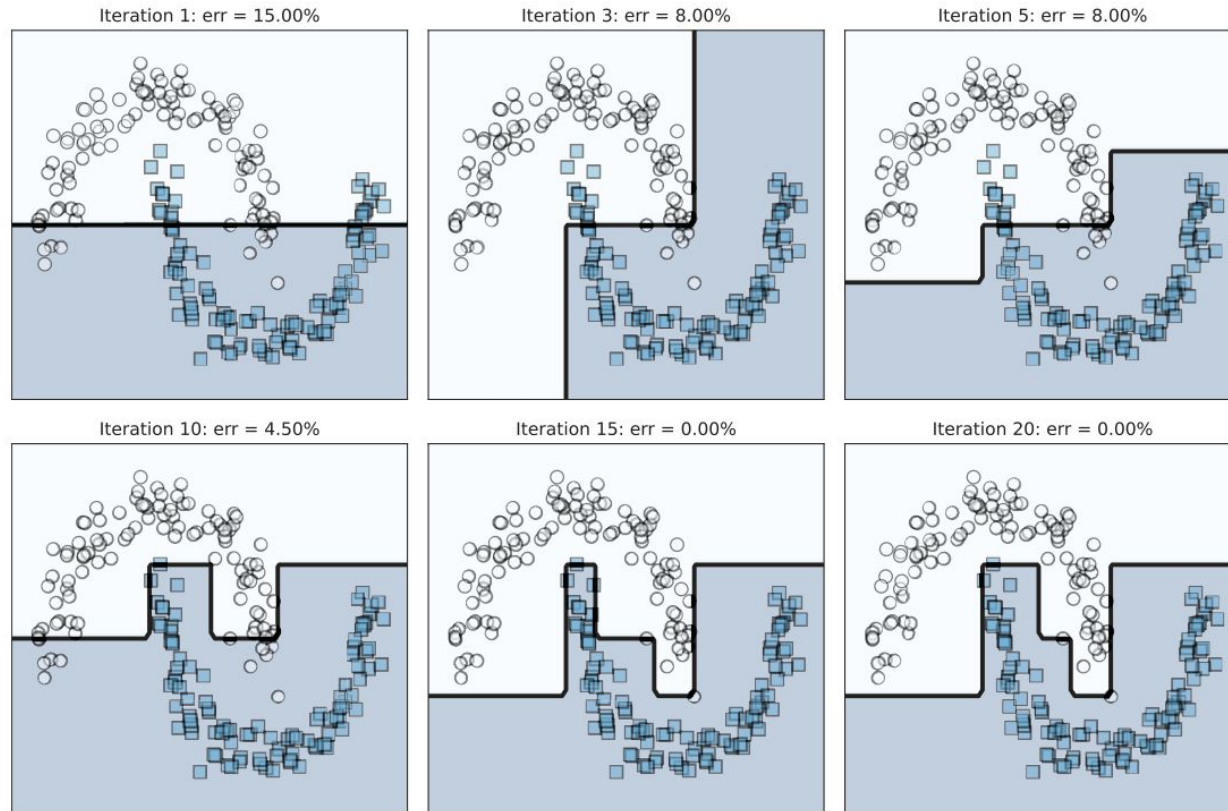
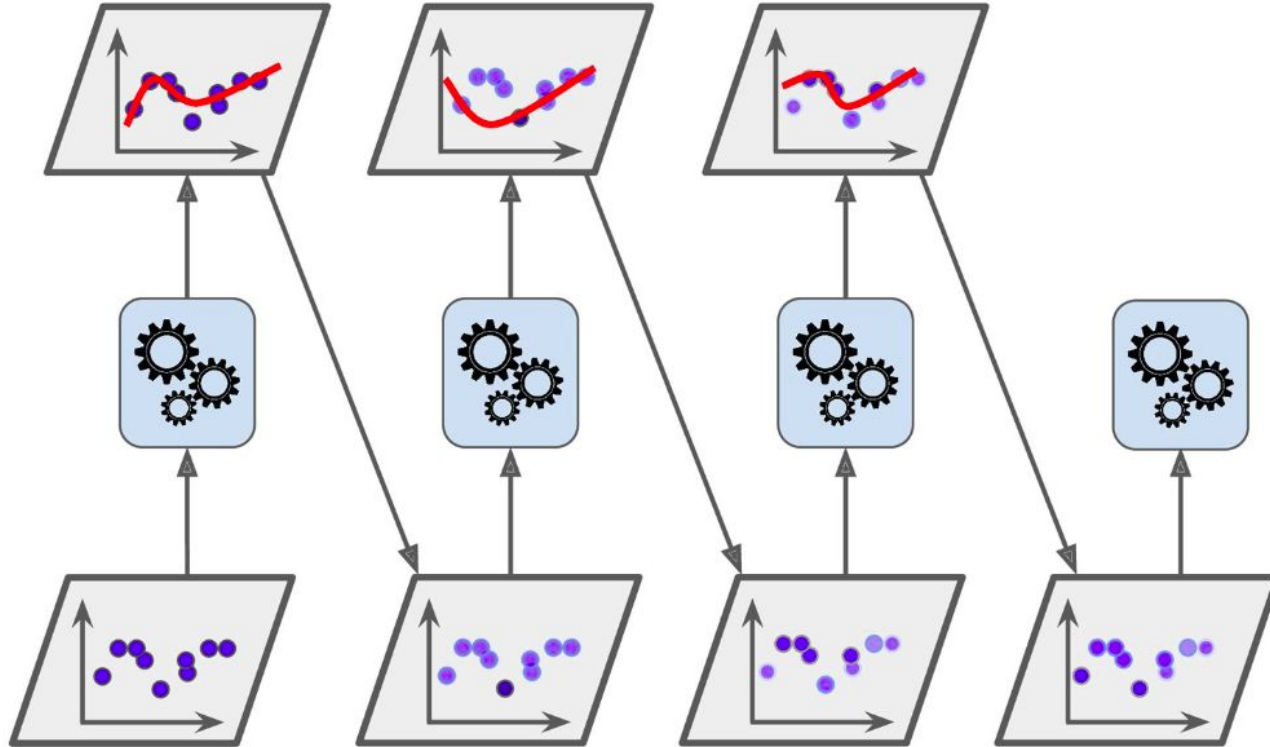


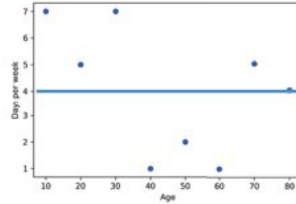
Figure 4.9 As the number of weak learners increases, the overall classifier is boosted into a strong model, which becomes increasingly nonlinear and is able to fit (and possibly overfit) the training data.

Boosting para regresión (esquema)

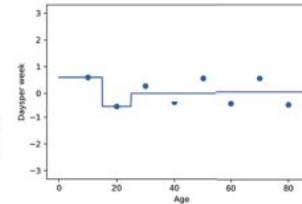
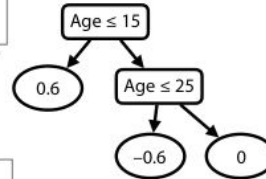


Boosting para regresión

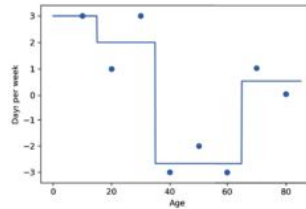
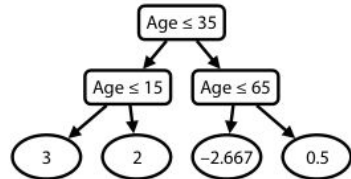
Weak learner 1



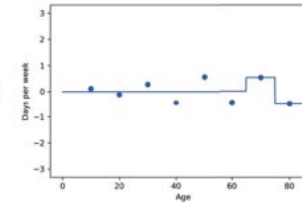
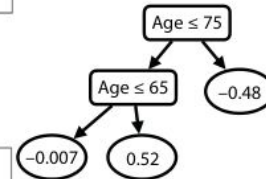
Weak learner 4



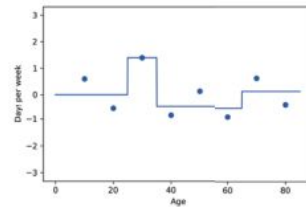
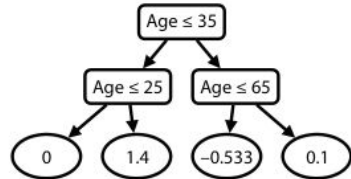
Weak learner 2



Weak learner 5



Weak learner 3



Los cinco modelos débiles (*weak learners*) en el modelo de *gradient boosting*. El primero es un **árbol de decisión de profundidad 0** que siempre predice el promedio de las etiquetas. Cada modelo débil sucesivo es un **árbol de decisión de profundidad máxima 2**, que ajusta los **residuales** de la predicción dada por los modelos débiles anteriores. Observa que las predicciones de los modelos débiles se hacen **más pequeñas**, porque los residuales se hacen más pequeños a medida que las predicciones del modelo fuerte (*strong learner*) se acercan más a las etiquetas.

Adaboost

AdaBoost fue el primer algoritmo de *boosting* realmente exitoso y marcó la pauta.

- **Mecanismo:** Se enfoca en ajustar los **pesos de las instancias (filas de datos)** en cada iteración.
 - Comienza dando el mismo peso a todas las instancias.
 - Tras cada modelo (o *weak learner*), aumenta el peso de las instancias que fueron **clasificadas incorrectamente** y disminuye el peso de las que fueron clasificadas correctamente.
- **Aprendices Débiles:** Típicamente utiliza **árboles de decisión muy poco profundos** (como los *decision stumps*, árboles de una sola división).
- **Predicción Final:** Es la **suma ponderada** de las predicciones de todos los aprendices débiles, donde los modelos que tuvieron mejor desempeño reciben un peso mayor.
- **Limitaciones:** Es sensible al **ruido** y a los **valores atípicos (outliers)**, ya que les asigna pesos muy altos para intentar corregir el error.

Decision stumps

Un decision stump es un árbol de decisión de 1 nodo raíz y 2 nodos hijos. Es un árbol de decisión muy simple. En el caso de tener una variable continua, en el nodo raíz tenemos un threshold, como en este caso.

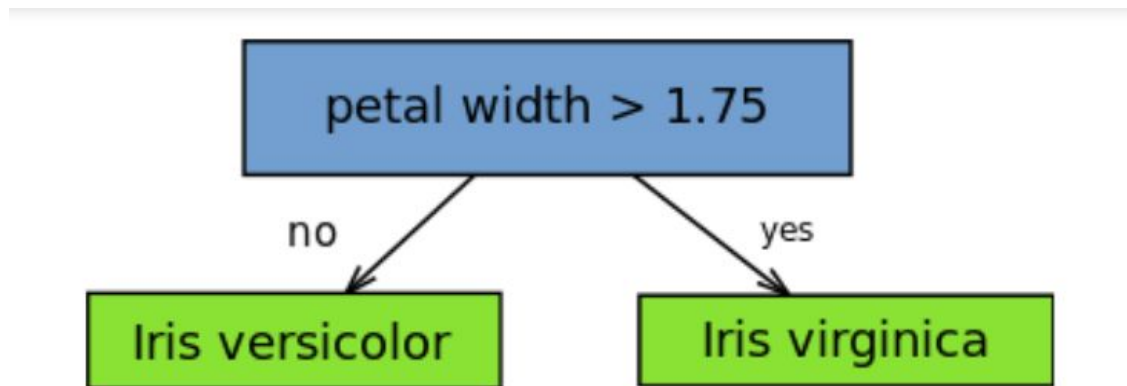


Tabla de datos

Dolor de pecho	Arterias bloqueadas	Peso	Enfermedad cardíaca
Si	Si	205	Si
No	Si	180	Si
Si	No	210	Si
Si	Si	167	Si
No	Si	156	No
No	Si	125	No
Si	No	168	No
Si	Si	172	No

Asignar los pesos

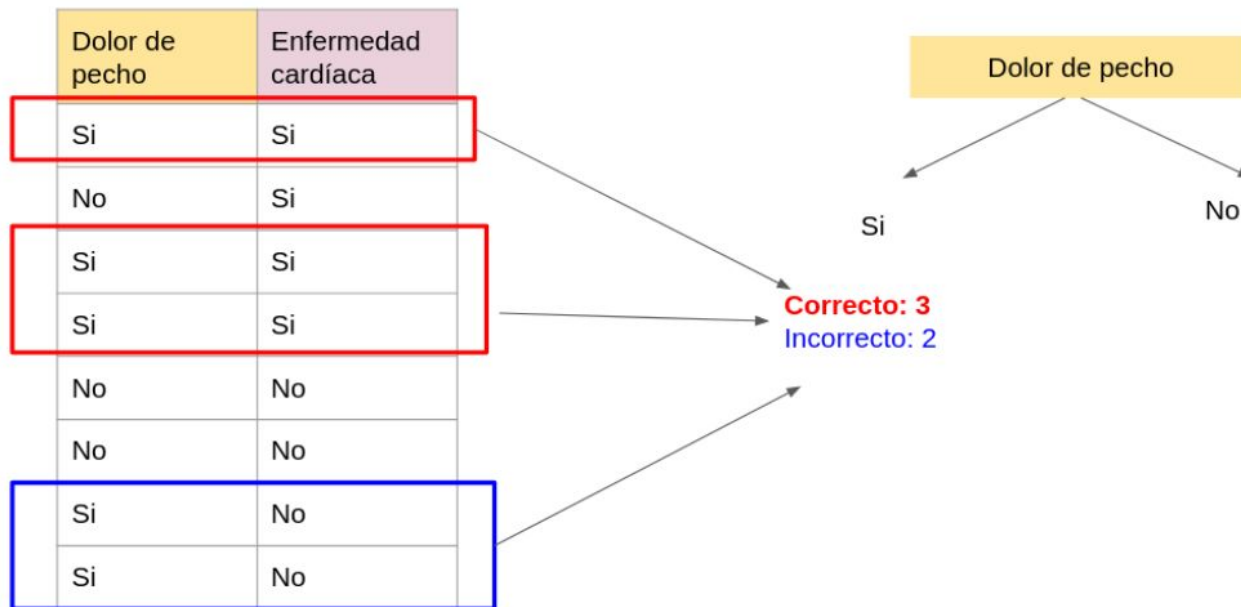
Vamos a asignar a cada muestra un peso.

Ese peso (weight) es $1/n$, en este caso, $1/8$

Dolor de pecho	Arterias bloqueadas	Peso	Enfermedad cardíaca	Weight (peso) del modelo
Si	Si	205	Si	1/8
No	Si	180	Si	1/8
Si	No	210	Si	1/8
Si	Si	167	Si	1/8
No	Si	156	No	1/8
No	Si	125	No	1/8
Si	No	168	No	1/8
Si	Si	172	No	1/8

Primer árbol de decisión

Primera rama del árbol



Primer árbol de decisión

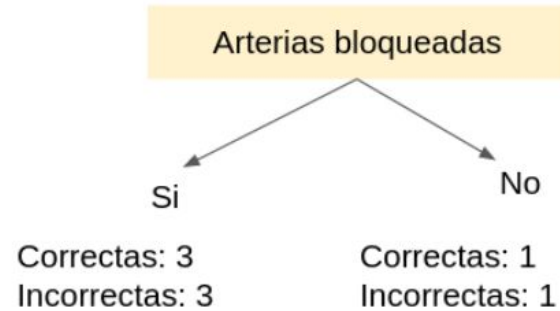
Segunda rama del árbol

Dolor de pecho	Enfermedad cardíaca
Si	Si
No	Si
Si	Si
Si	Si
No	No
No	No
Si	No
Si	No



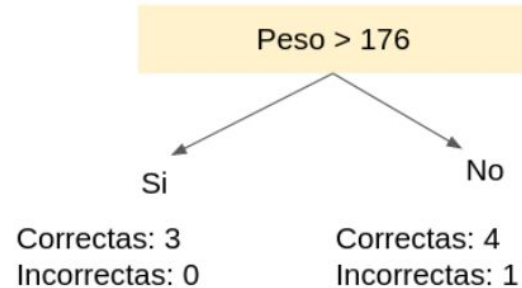
Segundo árbol de decisión

Arterias bloqueadas	Enfermedad cardíaca
Si	Si
Si	Si
No	Si
Si	Si
Si	No
Si	No
No	No
Si	No

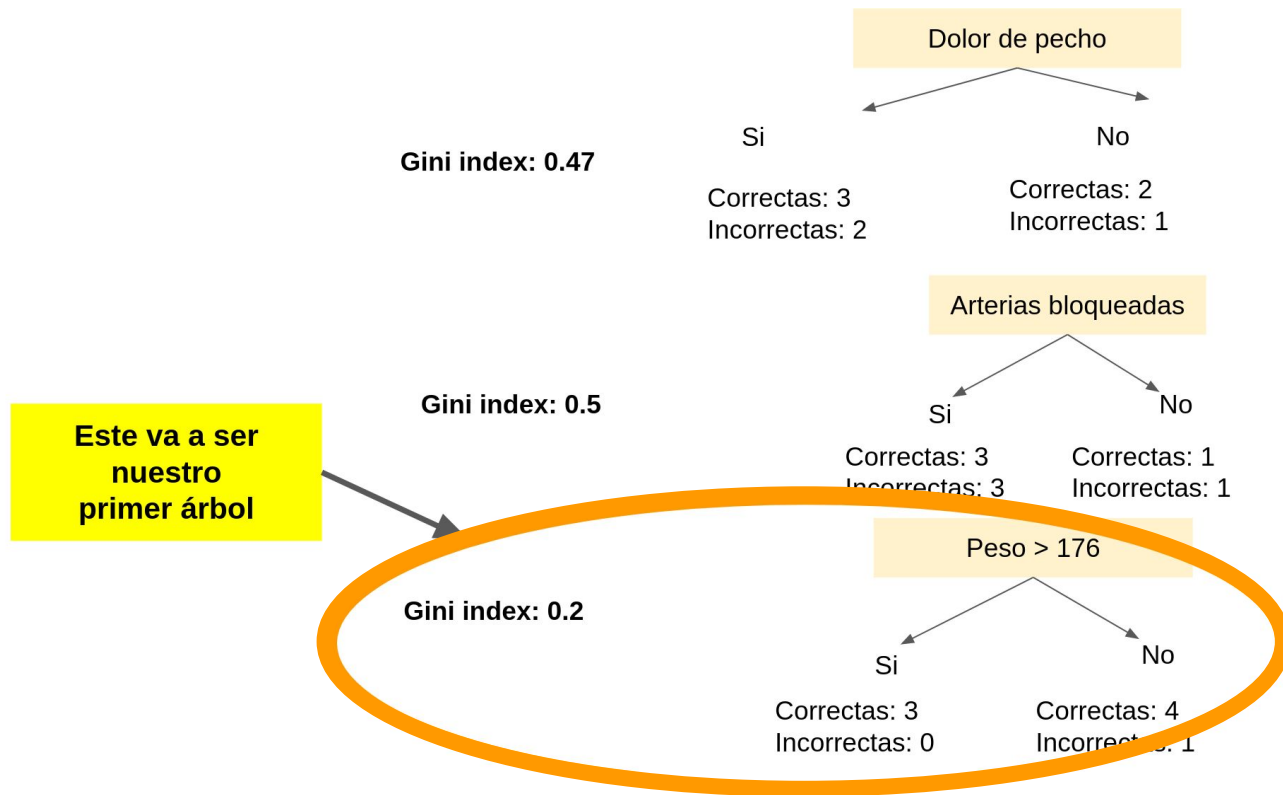


Tercer árbol de decisión

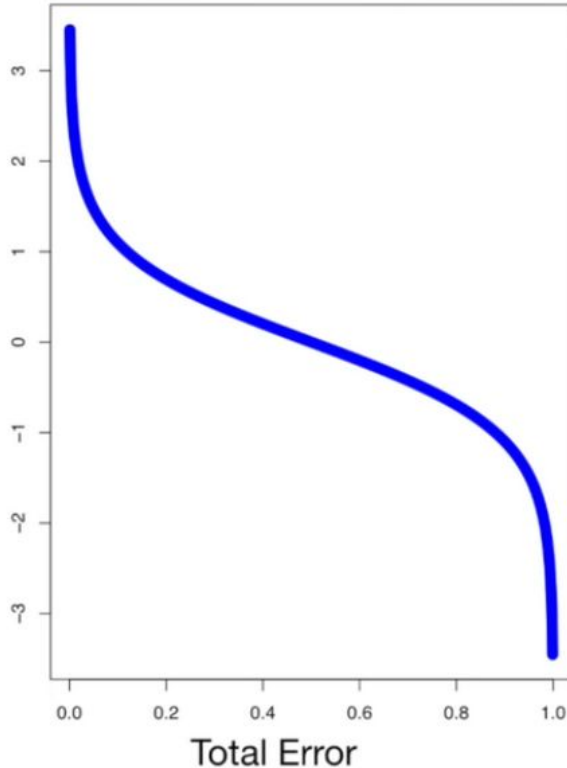
Peso	Enfermedad cardíaca
205	Si
180	Si
210	Si
167	Si
156	No
125	No
168	No
172	No



Calculamos el índice de Gini de cada árbol de decisión



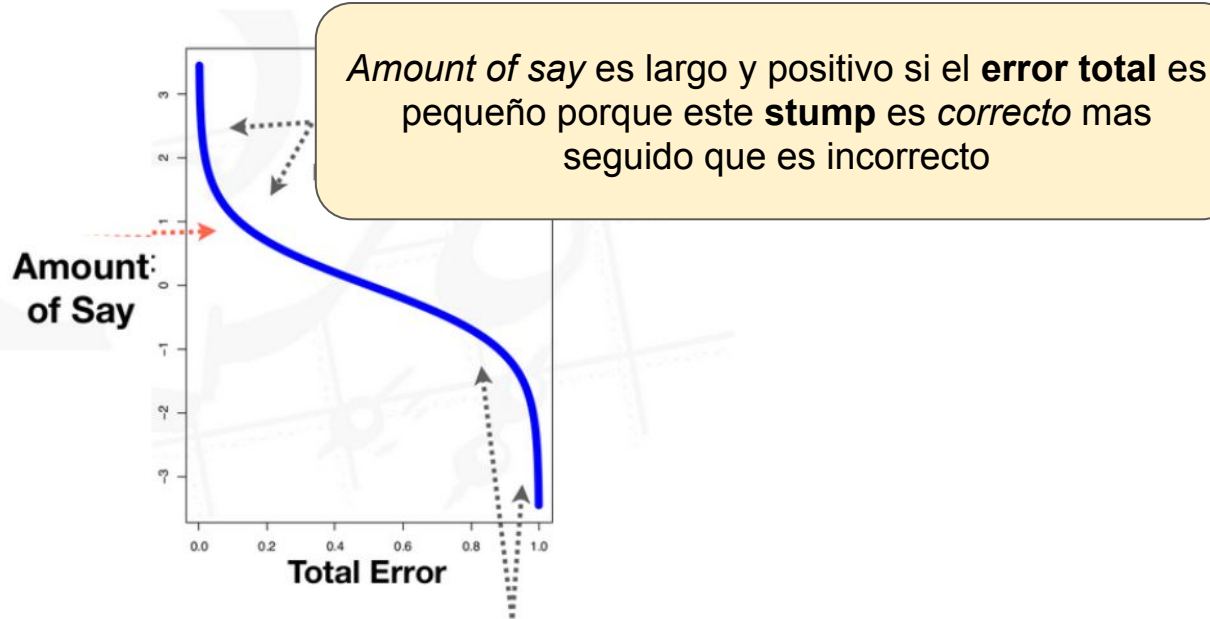
Cómo asignar las importancias



Este gráfico es la herramienta que utiliza AdaBoost para decidir si debe confiar mucho, poco o nada en las predicciones de cada stump individual según su precisión anterior.

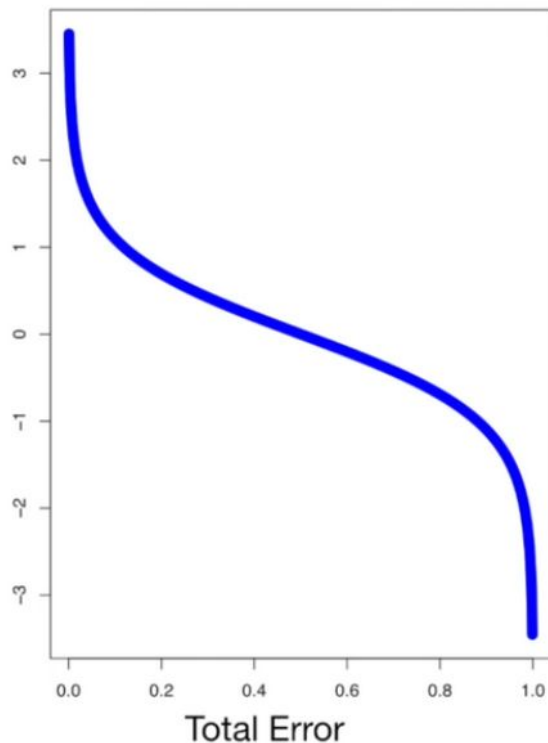
$$\text{Amount of Say} = \frac{1}{2} \log\left(\frac{1 - \text{Total Error}}{\text{Total Error}}\right)$$

Cómo interpretar este gráfico

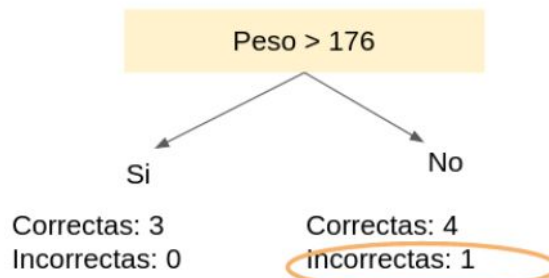


y el **Amount of Say** es largo y negativo si el **error total** es alto porque el stump es incorrecto más seguido de lo que es correcto. El **Amount of Say** negativo invierte la salida del **stump**

Primer árbol de decisión



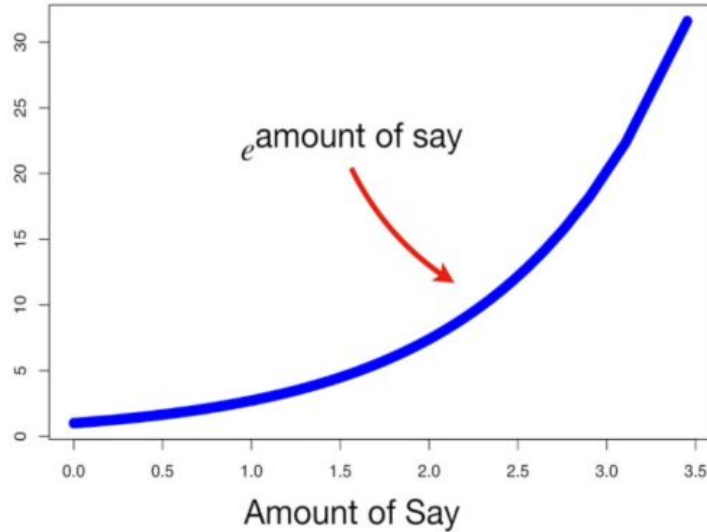
$$\text{Amount of Say} = \frac{1}{2} \log\left(\frac{1 - \text{Total Error}}{\text{Total Error}}\right)$$



$$\text{Amount of Say} = \frac{1}{2} \log(7) = 0.97$$

Cuando hay un error

New Sample Weight = sample weight $\times e^{\text{amount of say}}$

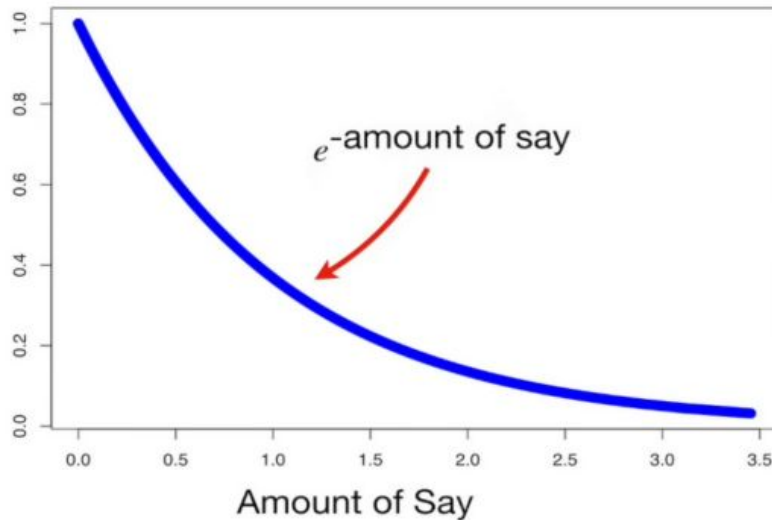


$$= \frac{1}{8} e^{\text{amount of say}}$$

$$= \frac{1}{8} e^{0.97} = \frac{1}{8} \times 2.64 = 0.33$$

Cuando hay un acierto

New Sample Weight = sample weight $\times e^{-\text{amount of say}}$



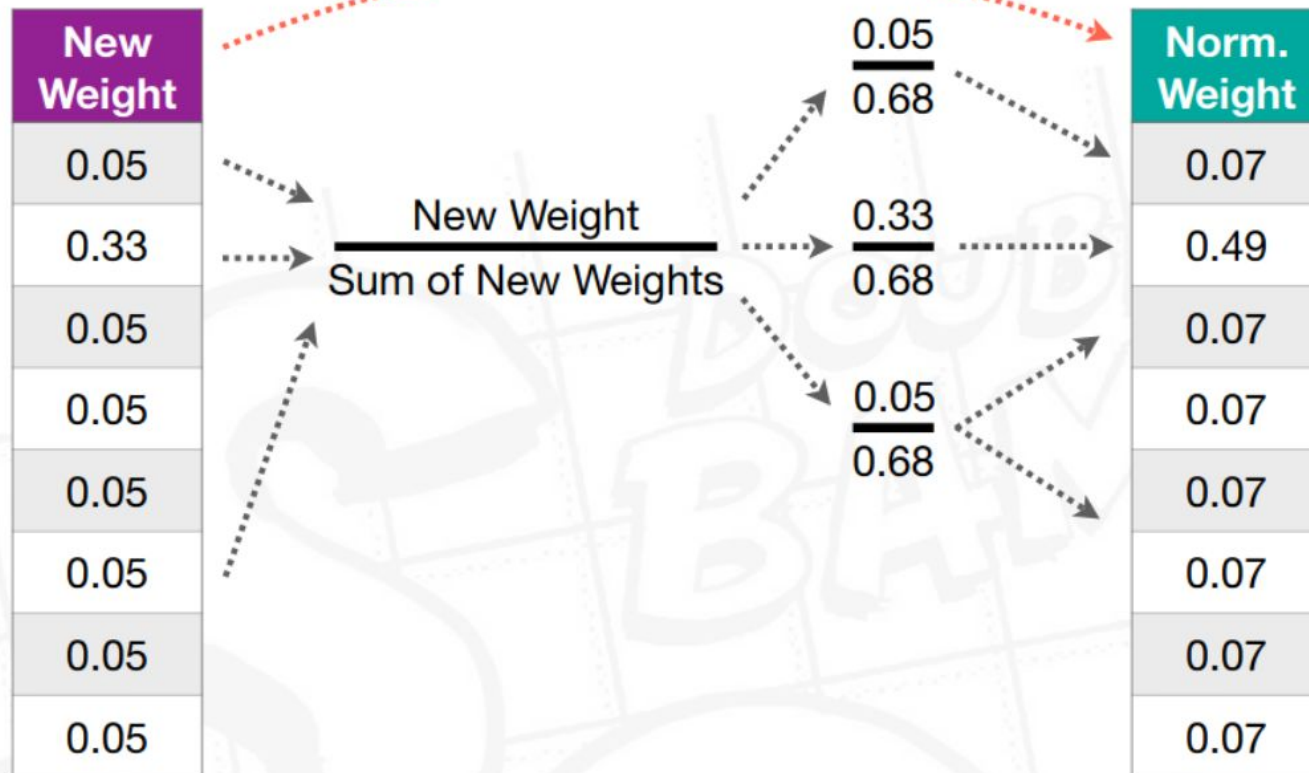
$$= \frac{1}{8} e^{-\text{amount of say}}$$

$$= \frac{1}{8} e^{-0.97} = \frac{1}{8} \times 0.38 = 0.05$$

Nuevos pesos

Dolor de pecho	Arterias bloqueadas	Peso	Enfermedad cardíaca	Weight (peso) del modelo	Nuevos pesos
Si	Si	205	Si	1/8	0.05
No	Si	180	Si	1/8	0.33
Si	No	210	Si	1/8	0.05
Si	Si	167	Si	1/8	0.05
No	Si	156	No	1/8	0.05
No	Si	125	No	1/8	0.05
Si	No	168	No	1/8	0.05
Si	Si	172	No	1/8	0.05

Normalizamos los pesos



Cómo influyen estos nuevos pesos

Pick a random number
between **0** and **1**...

**Sample
Weight**

0.07

0.49

0.07

0.07

0.07

0.07

0.07

0.07

...if the number is between **0** and **0.07**,
we add this sample to the new dataset...

...if the number is between
0.07 and **0.56** (because **$0.07 + 0.49 = 0.56$**), we add this
sample to the new dataset...

...if the number is between **0.56**
and **0.63** (because **$0.07 + 0.49 + 0.07 = 0.63$**), we add this sample
to the new dataset...etc.

New Dataset for the Next Stump

Chest Pain	Mass (kg)	Blocked Arteries	Heart Disease
Yes	81.7	Yes	Yes
Yes	68.6	Yes	Yes
No	88.8	Yes	Yes
Yes	68.6	Yes	Yes
Yes	69.5	Yes	No
No	74.8	Yes	No
Yes	68.6	Yes	Yes
Yes	68.6	Yes	Yes

Si una muestra tuviera un mayor peso (weight) se volvería a repetir varias veces en el nuevo dataset.

1. Asignamos los pesos ($1/n$) a cada muestra.
2. Creamos decision stumps para cada variable del dataset.
3. Calculamos el índice de Gini.
4. Según el error se calcula el Amount of Say.
5. Recalculamos los pesos.
6. Se repite el proceso.
7. Se suman todas las predicciones de todos los decision stumps.

- 1 Let one class be represented with a value of $+1$ and the other with a value of -1
- 2 Let each sample have the same starting weight ($1/n$)
- 3 **for** $k = 1$ **to** K **do**
- 4 Fit a weak classifier using the weighted samples and compute the k th model's misclassification error (err_k)
- 5 Compute the k th stage value as $\ln((1 - err_k) / err_k)$.
- 6 Update the sample weights giving more weight to incorrectly predicted samples and less weight to correctly predicted samples
- 7 **end**
- 8 Compute the boosted classifier's prediction for each sample by multiplying the k th stage value by the k th model prediction and adding these quantities across k . If this sum is positive, then classify the sample in the $+1$ class, otherwise the -1 class.

Algorithm 14.2: AdaBoost algorithm for two-class problems

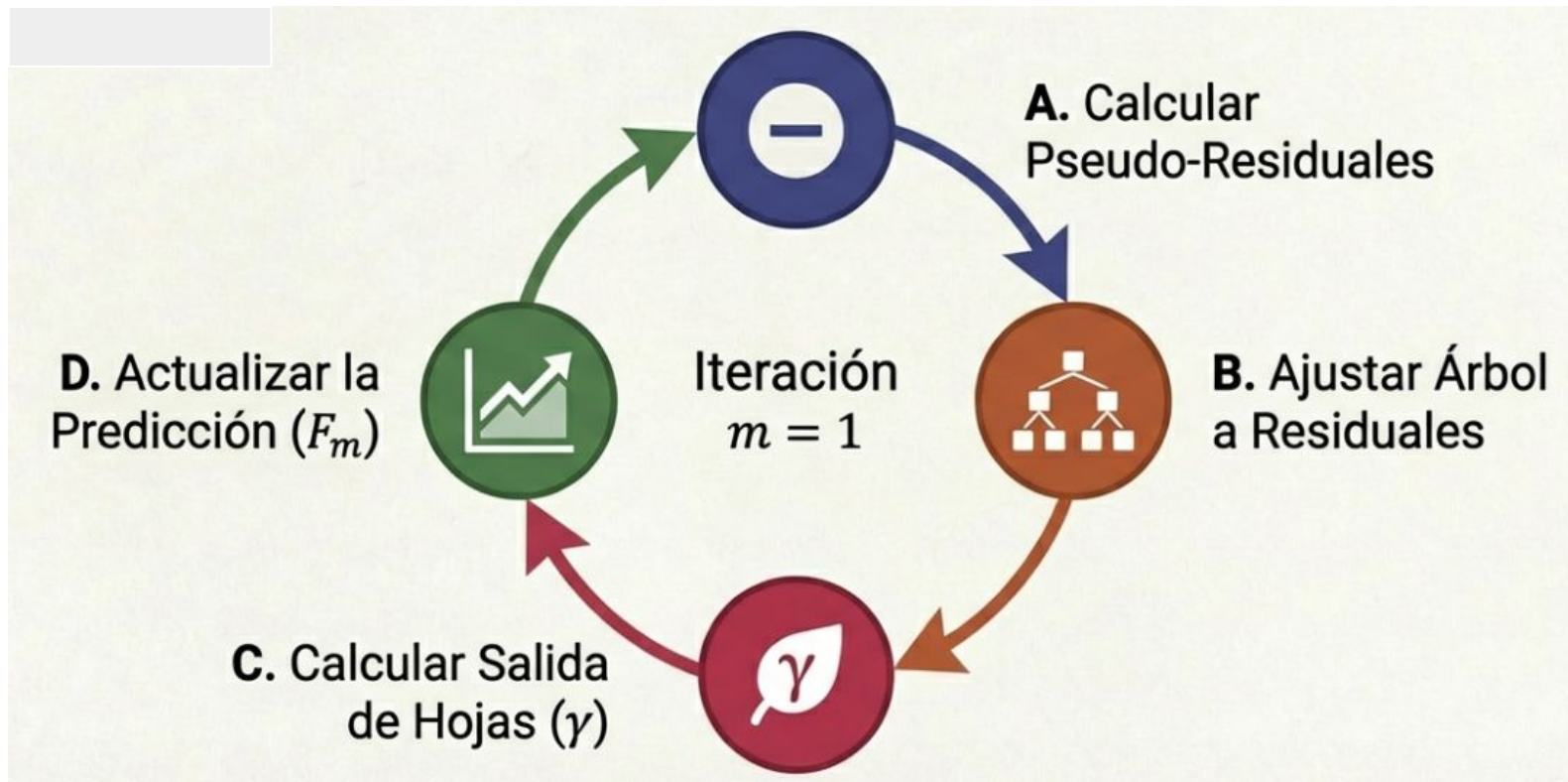
Gradient Boosting Machines (GBM)

Gradient boosting machines (GBM)

GBM es una generalización más flexible y moderna del *boosting*.

- **Mecanismo:** En lugar de manipular los pesos de las instancias (como AdaBoost), GBM entrena cada nuevo modelo para predecir el **error residual** (la diferencia) del modelo de conjunto actual. Utiliza el concepto de **Descenso de Gradiente** para minimizar la función de pérdida del modelo:
 - Calcula el gradiente negativo de la función de pérdida (que son esencialmente los residuales/errores en las tareas de regresión).
 - Entrena el nuevo árbol para predecir ese gradiente negativo.
 - Añade la predicción del nuevo árbol al modelo existente, pero escalada por un **ritmo de aprendizaje (*learning rate*)** para controlar la velocidad y evitar el sobreajuste.
- **Ventaja:** Permite utilizar **cualquier función de pérdida diferenciable**, lo que lo hace mucho más flexible para diversos problemas (regresión, clasificación, ranking, etc.).
- **Desventaja:** Su naturaleza **secuencial** lo hace lento y poco escalable con grandes conjuntos de datos.

Pasos de Gradient Boosting Machines



Veamos un caso paso a paso

Estamos ante un caso de regresión

Altura	Color	Género*	Peso
1.6	Azul	M	88
1.6	Verde	F	76
1.5	Azul	F	56
1.8	Rojo	M	73
1.5	Verde	M	77
1.4	Azul	F	57

Vamos a calcular la media aritmética

Media aritmética:
71.2 kg

Altura	Color	Género	Peso
1.6	Azul	M	88
1.6	Verde	F	76
1.5	Azul	F	56
1.8	Rojo	M	73
1.5	Verde	M	77
1.4	Azul	F	57

Calculamos los pseudo-residuales

$$\text{Residual} = \text{Peso Observado} - \text{Predicción Promedio}$$
$$16.8 = 88 - 71.2$$

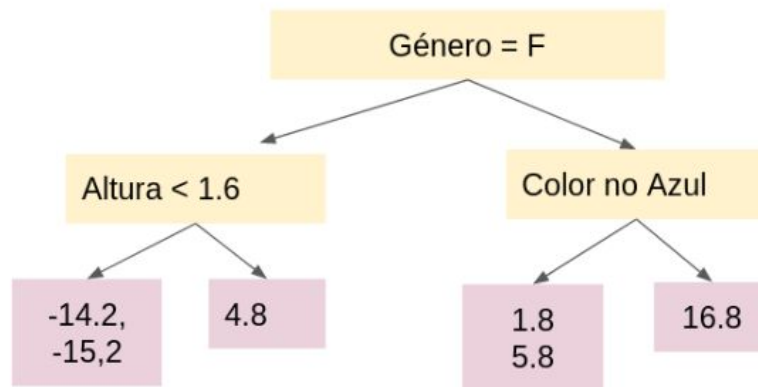
Media aritmética:
71.2 kg

Altura	Color	Género	Peso	Pseudo residuales
1.6	Azul	M	88	16.8
1.6	Verde	F	76	4.8
1.5	Azul	F	56	-15.2
1.8	Rojo	M	73	1.8
1.5	Verde	M	77	5.8
1.4	Azul	F	57	-14.2

Primer árbol de decisión

Uso los residuales en vez del valor verdadero

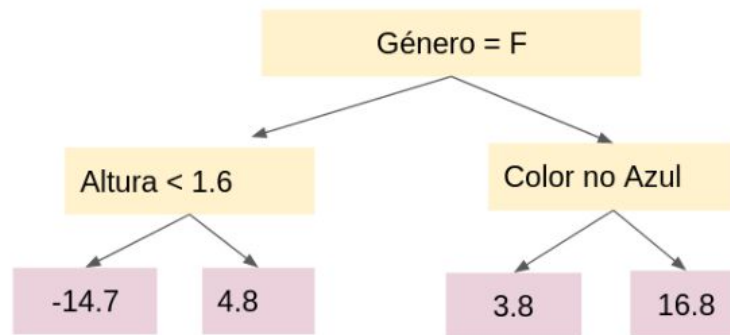
Altura	Color	Género	Res
1.6	Azul	M	16.8
1.6	Verde	F	4.8
1.5	Azul	F	-15.2
1.8	Rojo	M	1.8
1.5	Verde	M	5.8
1.4	Azul	F	-14.2



Primer árbol de decisión

Los valores que estan en la misma rama, los promediamos

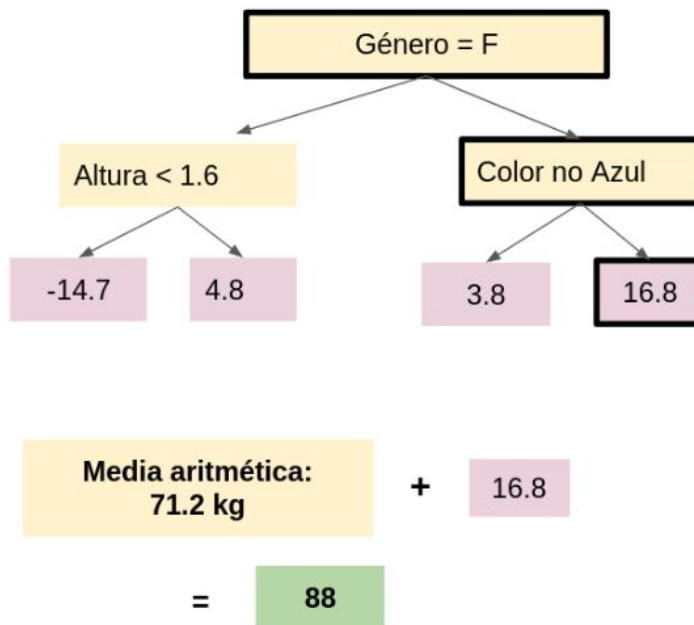
Altura	Color	Género	Res
1.6	Azul	M	16.8
1.6	Verde	F	4.8
1.5	Azul	F	-15.2
1.8	Rojo	M	1.8
1.5	Verde	M	5.8
1.4	Azul	F	-14.2



Primer árbol de decisión

Si a la media aritmética de todas las muestras, le sumo el valor del residual, vemos q coincide con el valor verdadero

Altura	Color	Género	Peso
1.6	Azul	M	88
1.6	Verde	F	76
1.5	Azul	F	56
1.8	Rojo	M	73
1.5	Verde	M	77
1.4	Azul	F	57



Mismo valor que el peso verdadero

Problema de overfitting

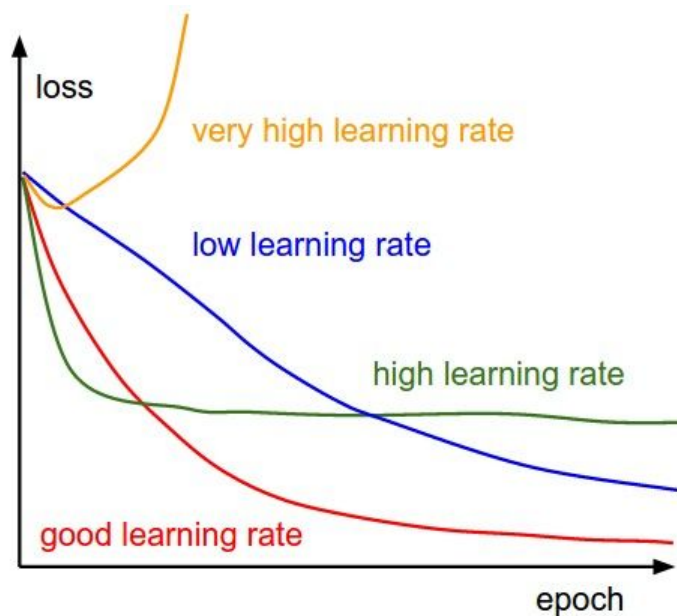
Predicción Inicial + Predicción del Árbol = Peso Final

71.2 (Promedio) + **16.8** (Residual) = **88.0** ✓ **Peso Real**
(Ajuste Exacto)



Memorizar los datos evita que el modelo
generalice para nuevas personas.

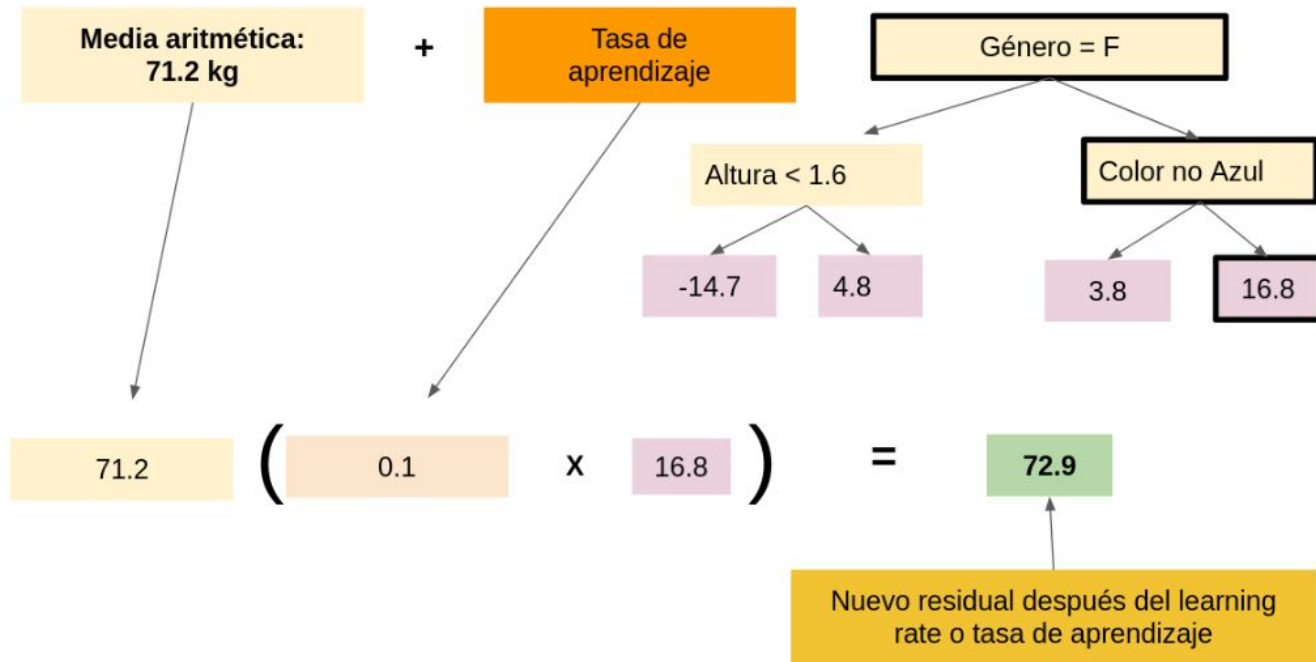
Recordemos el concepto de tasa de aprendizaje



Es importante definir una tasa de aprendizaje óptima, ya que si ese valor es muy grande: el algoritmo no converge, pero si es muy pequeño: vamos a necesitar muchos pasos o épocas para la convergencia.

Agregamos la tasa de aprendizaje

En este caso, definimos, una tasa de aprendizaje igual a 0.1.



Para evitar el sobreajuste, escalamos la contribución del nuevo árbol usando una **Tasa de Aprendizaje** (Learning Rate).

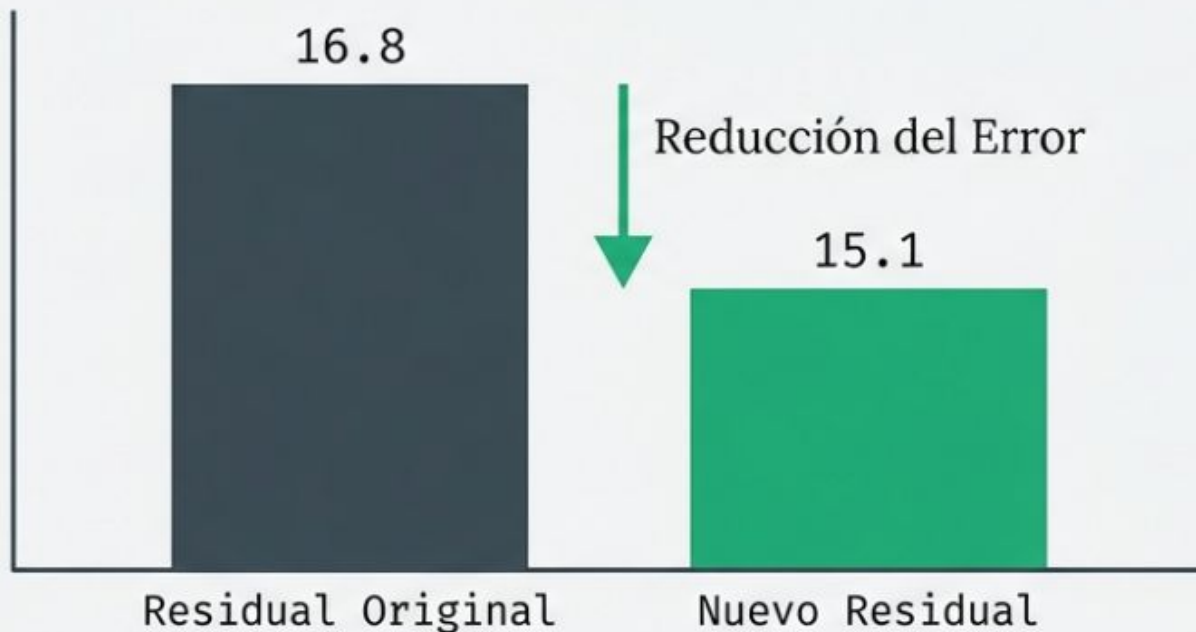
$$71.2 + (0.1 \times 16.8) = 72.9$$

Tasa de Aprendizaje



Un pequeño paso en la dirección correcta

Disminución de residuales



$$\text{Nuevo Residual} = 88 \text{ (Real)} - 72.9 \text{ (Predicho)} = 15.1$$

Los residuales disminuyen

Los residuales han disminuido



Residuales
que surgen
de la resta

Residuales luego de agregar la tasa
de aprendizaje

Peso	residuales 1	residuales 2	residuales 3
88	16.8	15.1	13.6
76	4.8	4.3	3.9
56	-15.2	-13.7	-12.4
73	1.8	1.4	1.1
77	5.8	5.4	-5.1
57	-14.2	-12.7	-11.4

Residuales luego de
agregar el 2do árbol de
decisión

**Los residuales han
disminuido,
resultando en una mejor
predicción del modelo**

El modelo final es una suma ponderada de la hoja inicial y todos los árboles subsecuentes.


$$71.2 + \left[0.1 \times 16.8 \right] + \left[0.1 \times 15.1 \right] = 74.4$$



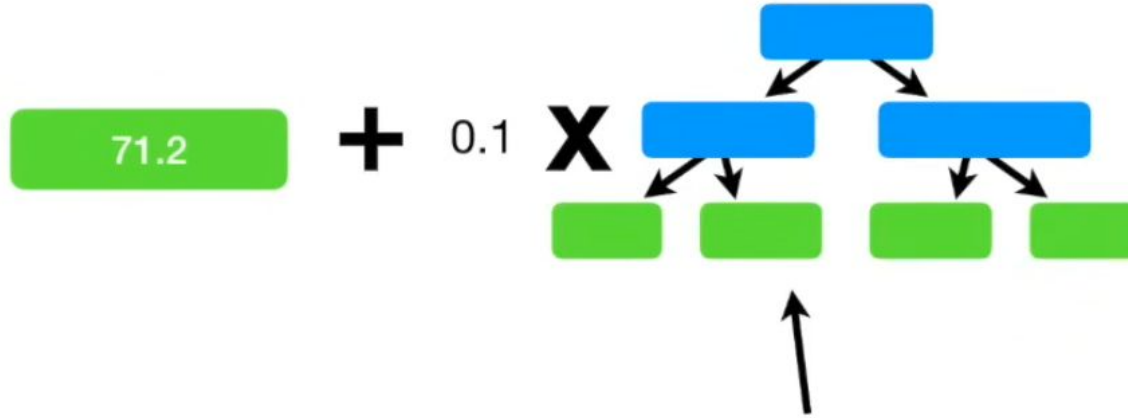
Resumen de los pasos

71.2

Empezamos con una hoja que tiene el valor medio de la variable que queremos **predecir**.

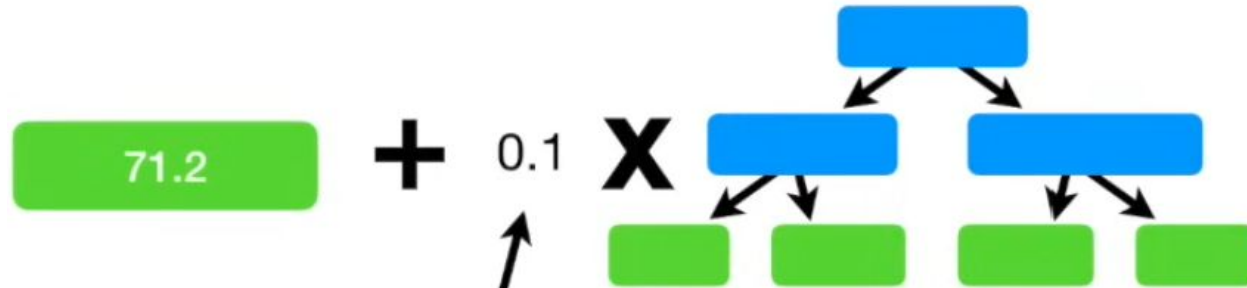
En este caso, queremos predecir la variable **Peso**

Resumen de los pasos



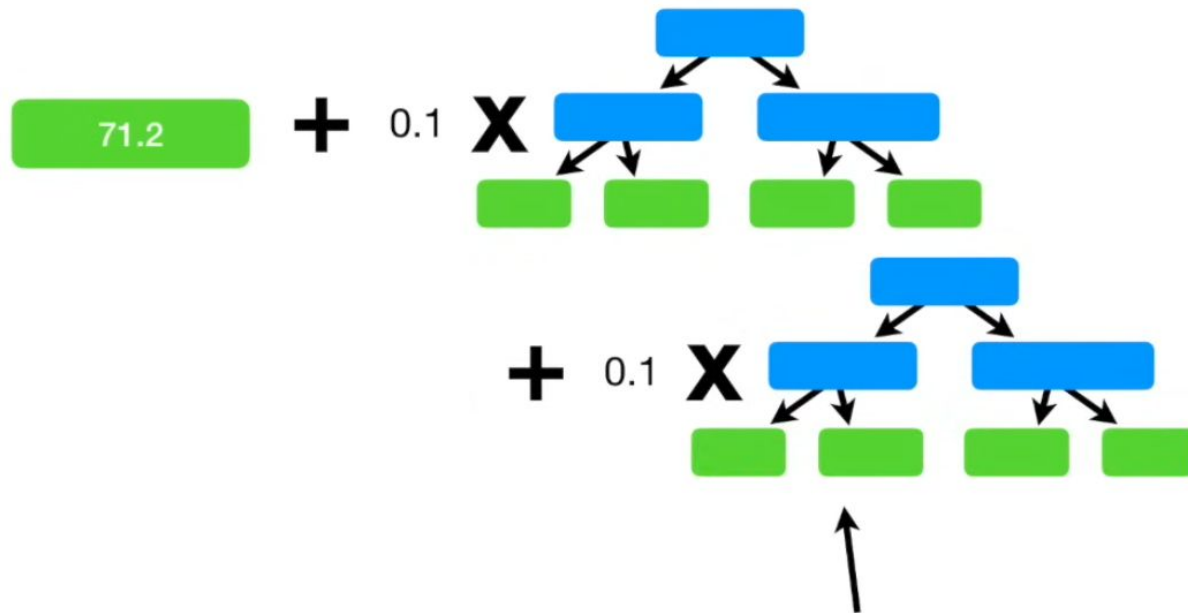
Luego, agregamos un árbol basado en los **residuales**, es decir, en la diferencia de los **valores observados** y los **predichos**

Resumen de los pasos



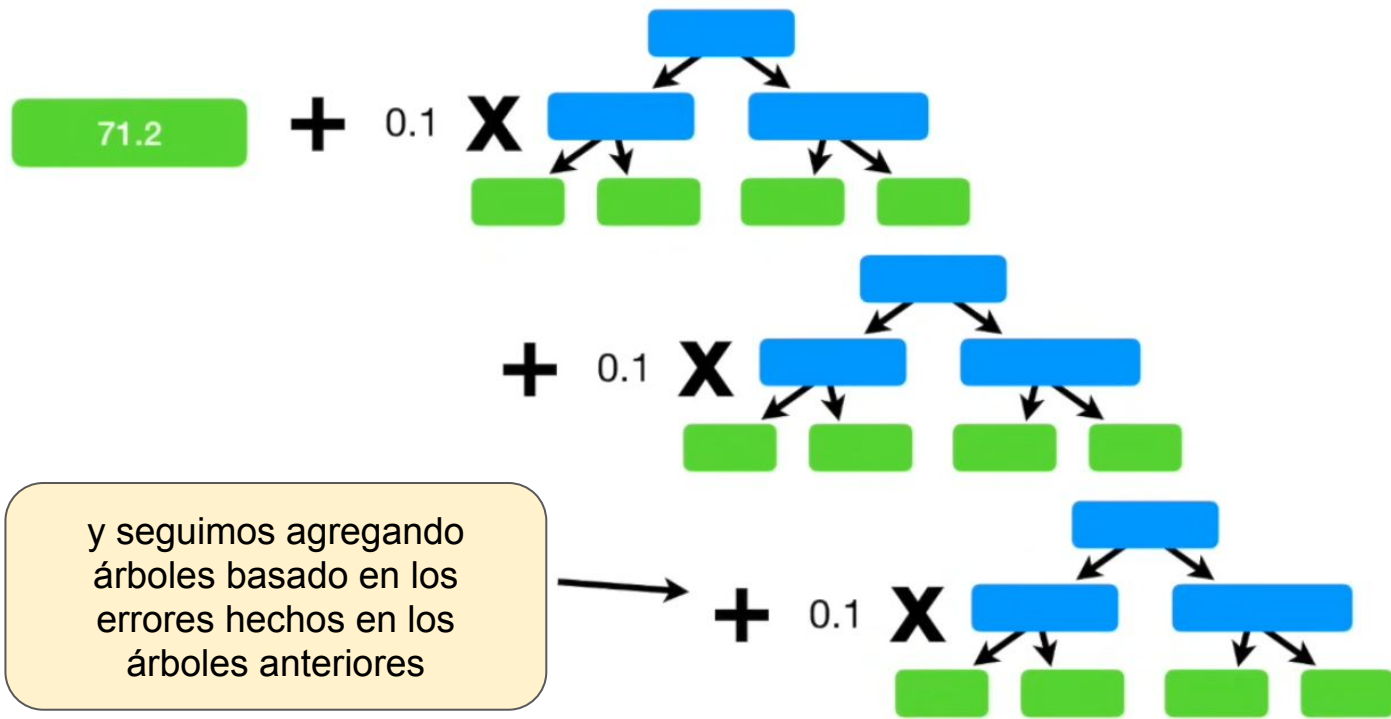
y escalamos la contribución de cada árbol a la **predicción** final con el **learning rate**

Resumen de los pasos



Luego, agregamos otro árbol basado en los nuevos **residuales**

Resumen de los pasos



Tasa de aprendizaje y número de árboles

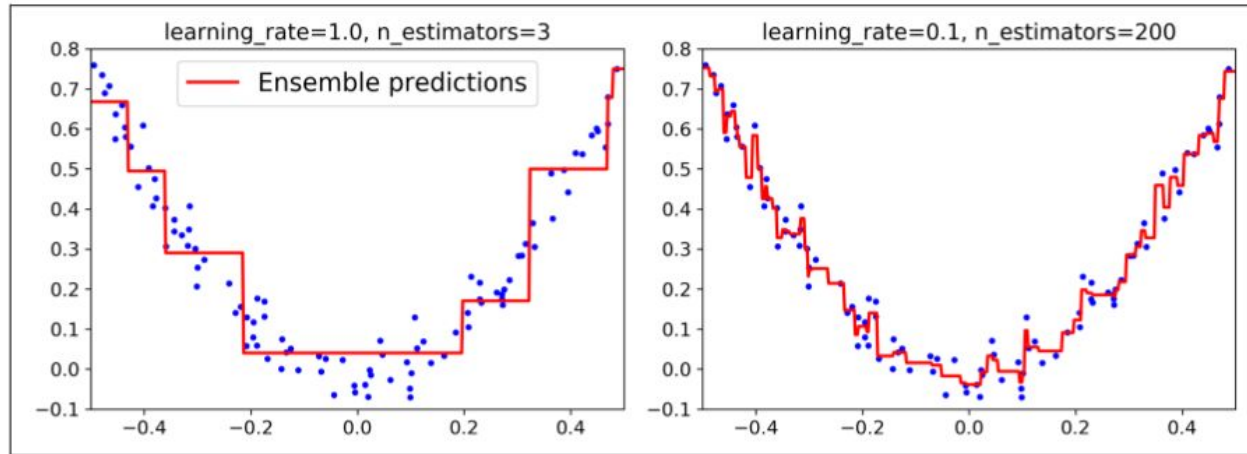
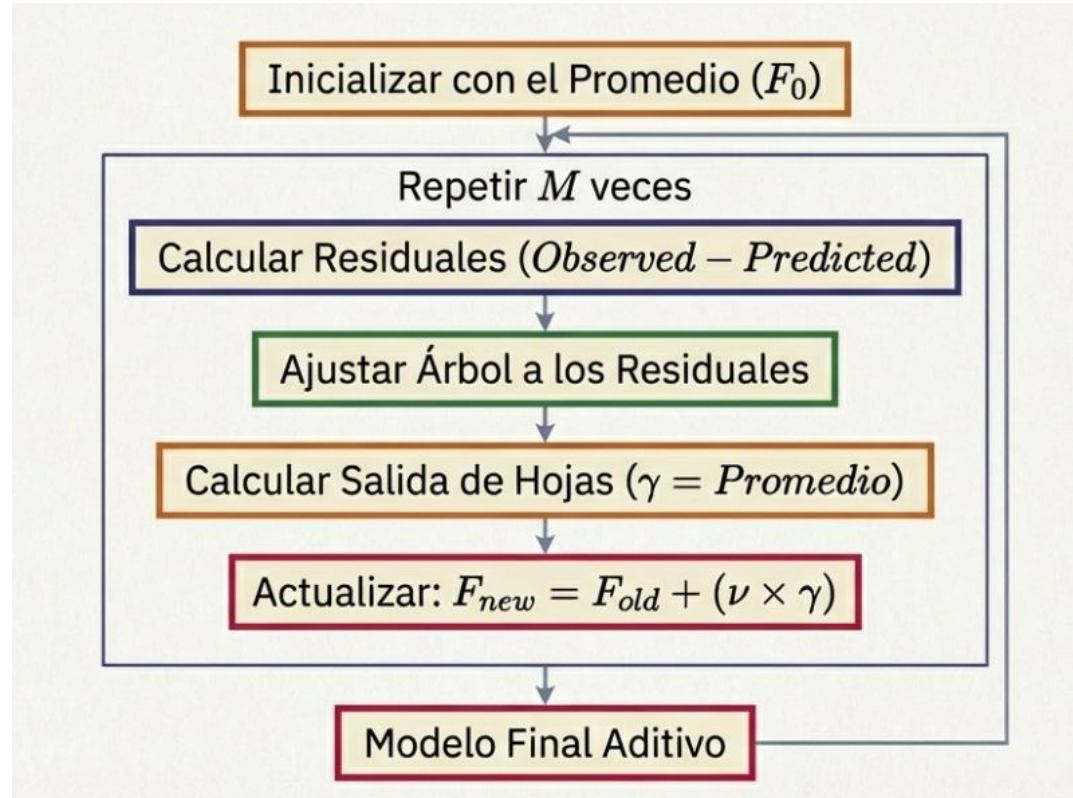


Figure 7-10. GBRT ensembles with not enough predictors (left) and too many (right)

Una tasa de aprendizaje más pequeña hace que cada nuevo árbol contribuya menos a la predicción final, lo que obliga al algoritmo a tomar más pasos (más árboles)

Cuanto más pequeño sea el learning rate del algoritmo, más árboles (n_estimators) vamos a necesitar para lograr resultados óptimos

Pasos del algoritmo



- 1 Select tree depth, D , and number of iterations, K
- 2 Compute the average response, $\bar{y}$, and use this as the initial predicted value for each sample
- 3 **for** $k = 1$ to K **do**
 - 4 Compute the residual, the difference between the observed value and the *current* predicted value, for each sample
 - 5 Fit a regression tree of depth, D , using the residuals as the response
 - 6 Predict each sample using the regression tree fit in the previous step
 - 7 Update the predicted value of each sample by adding the previous iteration's predicted value to the predicted value generated in the previous step
- 8 **end**

Algorithm 8.3: Simple Gradient Boosting for Regression

XGBoost

XGBoost (eXtreme Gradient Boosting)

XGBoost es una implementación avanzada de GBM conocida por su **velocidad, rendimiento y regularización**.

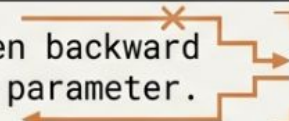
Regularización: Incorpora términos de regularización **L1 (Lasso) y L2 (Ridge)** en la función objetivo, lo que lo hace más robusto contra el **sobreajuste** que el GBM clásico.

Paralelización: A diferencia de GBM, XGBoost puede utilizar **paralelización** para la construcción de árboles, acelerando significativamente el proceso de entrenamiento.

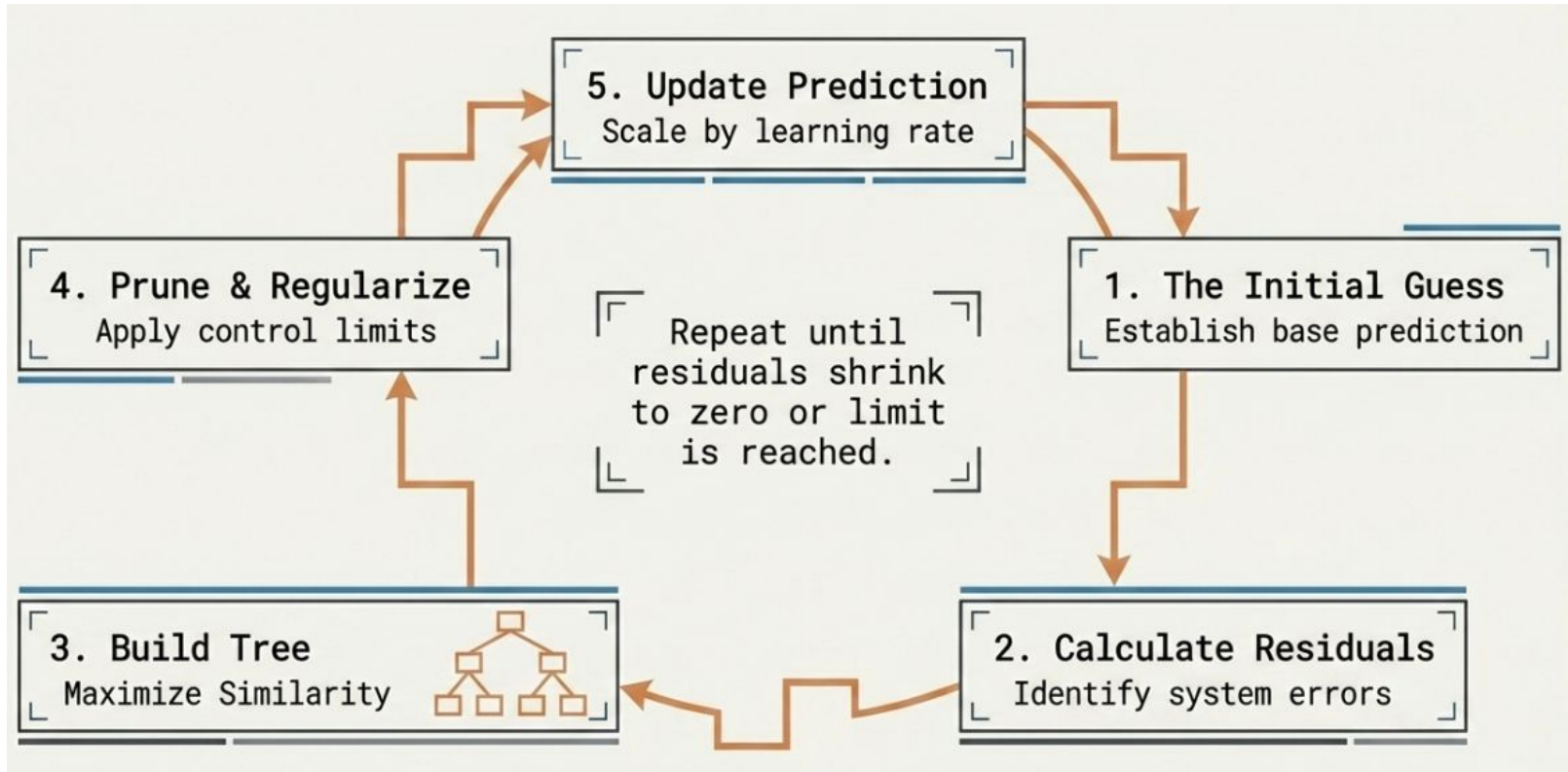
Manejo de valores faltantes: Puede aprender internamente el mejor camino para los valores faltantes (*missing values*) sin necesidad de imputación previa.

Crecimiento de árboles: Utiliza un crecimiento de árboles **nivel-por-nivel** (*level-wise*), lo que facilita la paralelización, pero puede ser menos eficiente en algunos casos.

XGBoost a diferencia de GBM

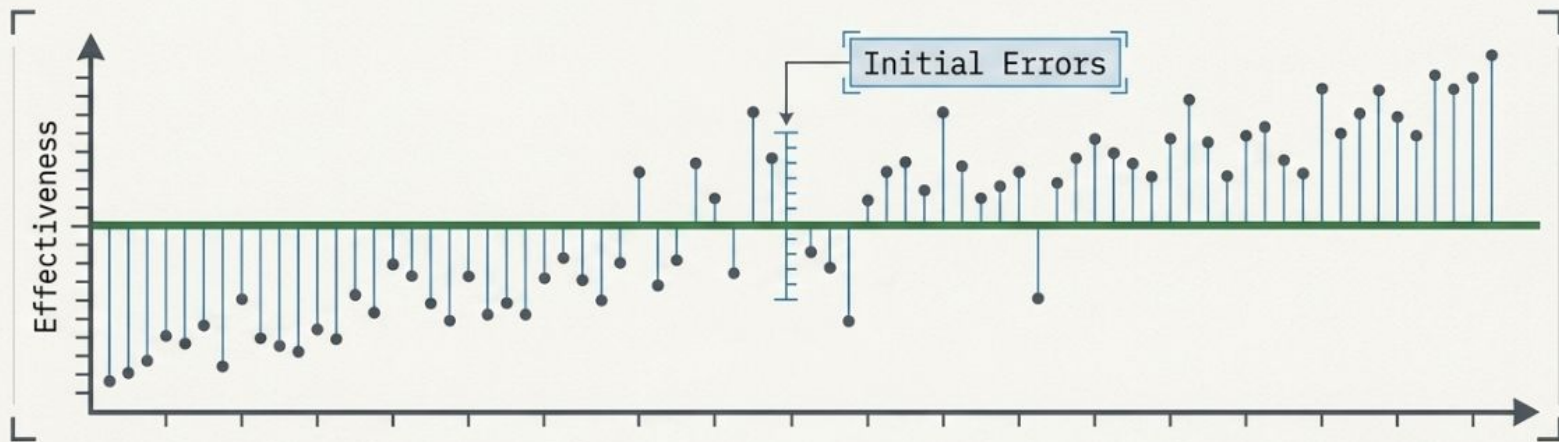
Standard Gradient Boost		XGBoost
Tree Building	Variance/Gini impurity algorithms.	Custom Similarity Scores via 2nd-order Taylor approximations (<i>G</i> & <i>H</i>).
Pruning	Forward stop-splitting criteria. 	Builds to max depth, then backward prunes using complexity parameter. 
Regularization	No native tree regularization.	Embeds L2 regularization directly into leaf calculations.
Missing Data	Requires external imputation. 	Native Sparsity-Aware Split Finding. 
System Speed	Standard memory allocation.	Cache-aware access, out-of-core computation, and quantile sketching.

Pasos de XGBoost



Paso 1: Inicializar en 0.5

All XGBoost models originate from a single, baseline assumption.



The Baseline Prediction

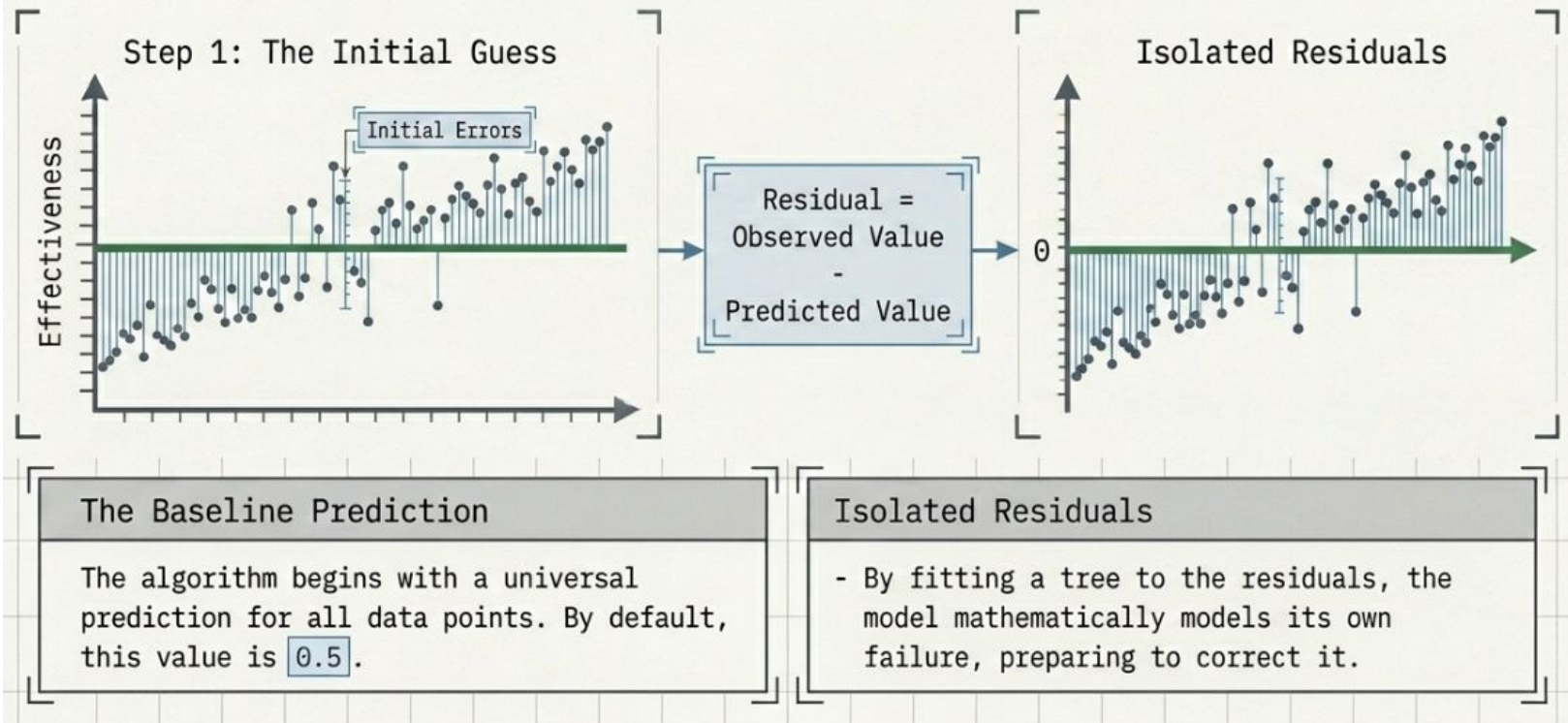
The algorithm begins with a universal prediction for all data points. By default, this value is 0.5.

Task Context

- Regression: Acts as a flat starting value.
- Classification: Translates to a base probability (log-odds of 0).

Paso 2: Separar los residuos

We stop predicting the target variable and strictly predict the mistakes.



Paso 3A: Dividir según Similarity Score

Unlike standard trees that minimize variance, XGBoost clusters data by maximizing similarity.

$$\text{Similarity Score} = \frac{(\sum \text{Residuals})^2}{\text{Number of Residuals} + \lambda}$$

Low Similarity

Leaf node

+7.5 -7.5

Math

$$(+7.5 + -7.5)^2 = 0$$

Divergent residuals cancel out before squaring, yielding a low score.

High Similarity

Leaf node

+7.5 +6.5

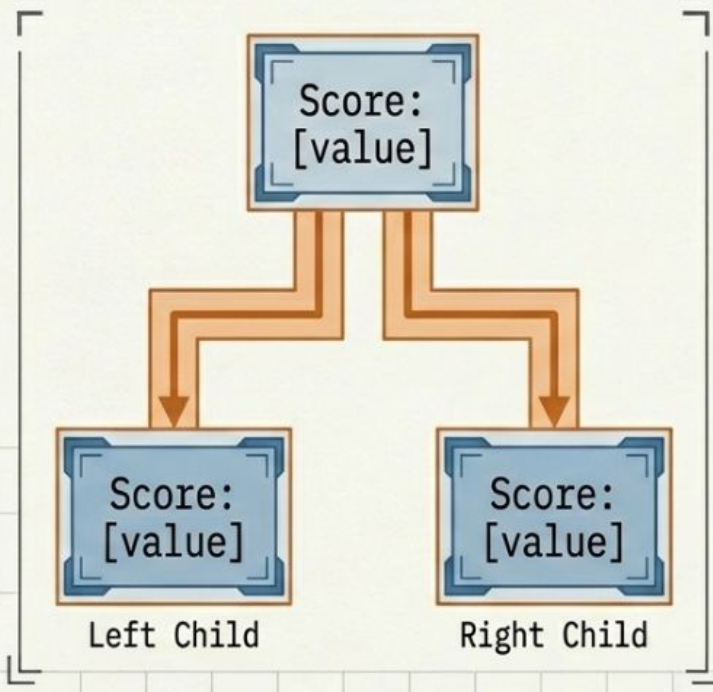
Math

$$(+7.5 + +6.5)^2 = 196$$

Similar residuals compound, yielding a massive score.

Parte 3B: Evaluando cada posible split

The algorithm calculates the improvement in purity after a split to determine the most effective branching decision.

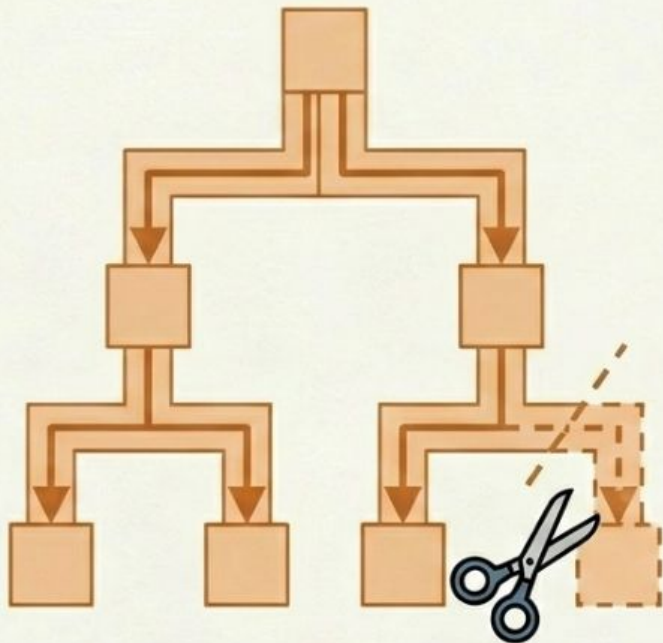


$$\text{Gain} = \text{Left Score} + \text{Right Score} - \text{Root Score}$$

1. The algorithm tests candidate thresholds.
2. It calculates the Similarity Score for the resulting leaves.
3. It determines the overall Gain of the split.
4. The threshold producing the highest positive Gain is locked in.

Paso 4: Backward pruning

Standard trees build downward to a threshold. XGBoost builds to maximum depth, then cuts its way backward.



The Pruning Logic (γ)

IF (Gain - Gamma) < 0 $\rightarrow$ Remove Branch.

IF (Gain - Gamma) > 0 $\rightarrow$ Keep Branch.

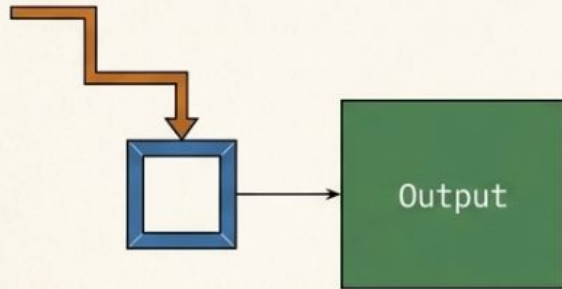
A branch with negative gain is kept if a branch further down the line proves highly valuable. Gamma (γ) serves as the user-defined complexity dial.

Paso 5: Valores de salida y regularización

Once the topology is locked, the tree calculates the actual prediction output for each leaf.

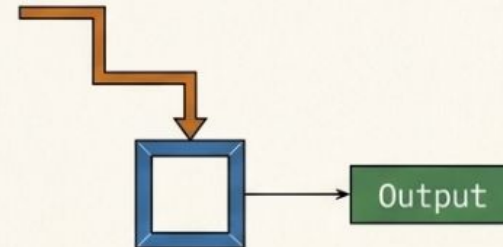
$$\text{Output} = \frac{\sum \text{Residuals}}{\text{Number of Residuals} + \lambda}$$

Without Regularization ($\lambda = 0$)



Output is exactly average.
Highly sensitive to outliers.

With Regularization ($\lambda = 1$)

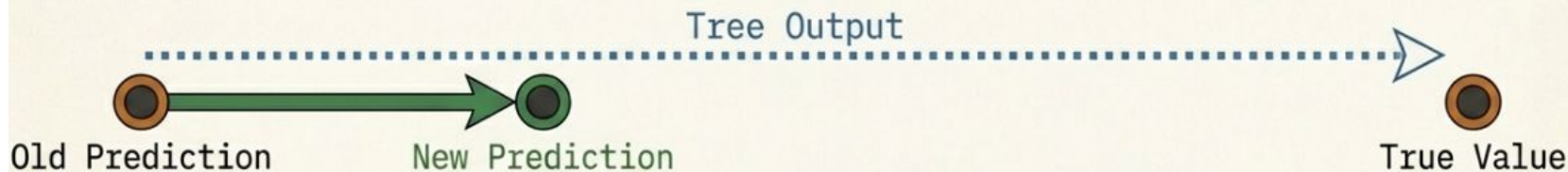


Lambda resides in the denominator. As it increases, the output value mathematically shrinks toward zero, reducing variance and preventing overfitting.

Paso 6: Actualizar las predicciones

Taking small, scaled steps in the right direction yields lower variance.

$$\text{New Prediction} = \text{Old Prediction} + (\eta * \text{Tree Output})$$

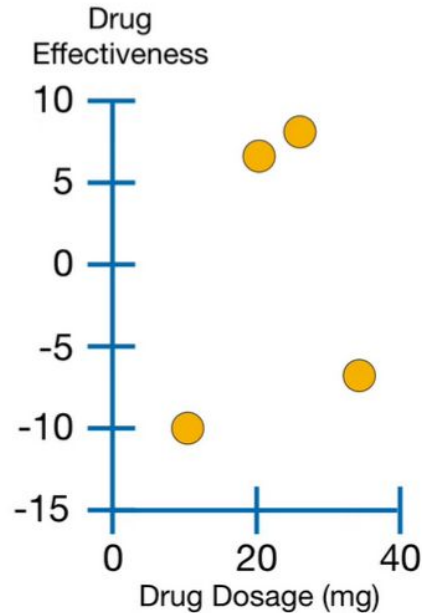


The Learning Rate

We do not blindly add the raw output of the new tree. We scale it using η (Eta, default 0.3) to take fractional steps. This prepares the system for the next cycle.

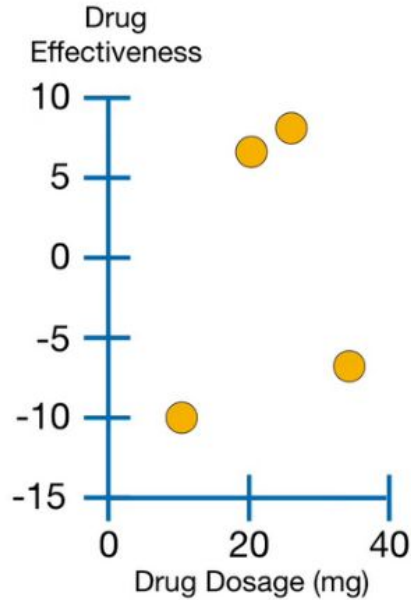
Veamos XGBoost paso a paso

Supongamos que nos interesa predecir la efectividad de una droga a partir de la dosis aplicada

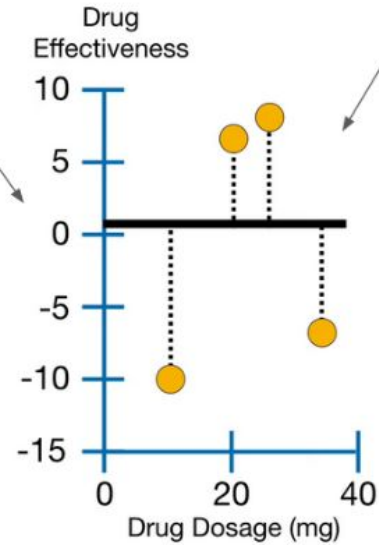


Baseline por default

Predicción por default: 0.5

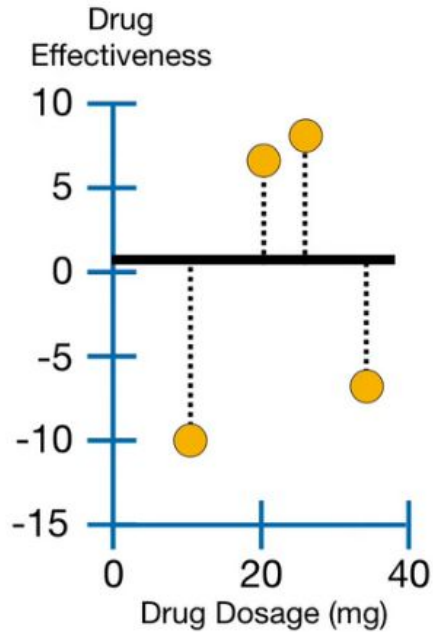


Calculo los
residuales

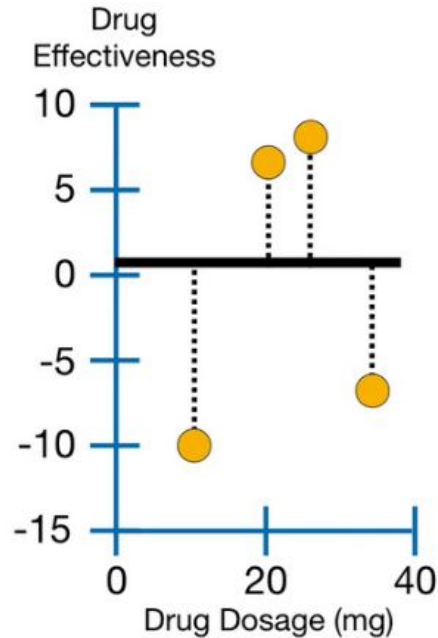


Residuales para cada predicción

-10.5, 6.5, 7.5, -7.5



Similarity Score



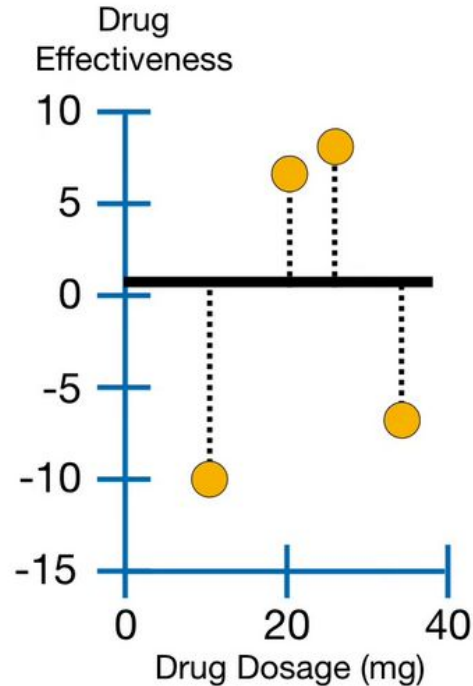
-10.5, 6.5, 7.5, -7.5

Vamos a calcular el Similarity Score para los residuales:

$$\text{Similarity Score} = \frac{\text{Sum of Residuals, Squared}}{\text{Number of Residuals} + \lambda}$$

Regularización

Fijamos en 0 la regularización



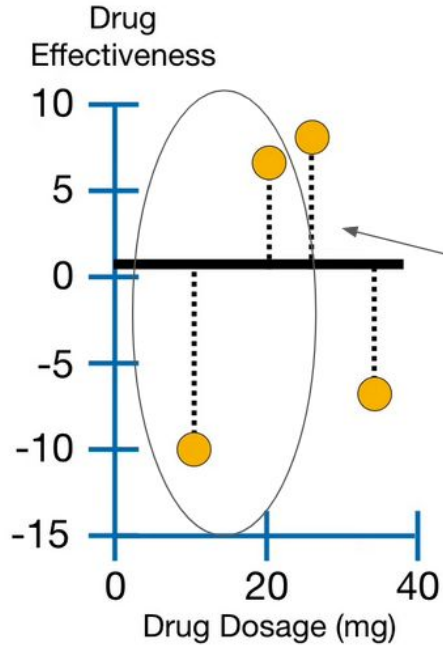
-10.5, 6.5, 7.5, -7.5

Vamos a calcular el Similarity Score para los residuales:

$$\text{Similarity Score} = \frac{(-10.5 + 6.5 + 7.5 + -7.5)^2}{4 + 0} = 4$$

Lambda = 0

Similarity Score



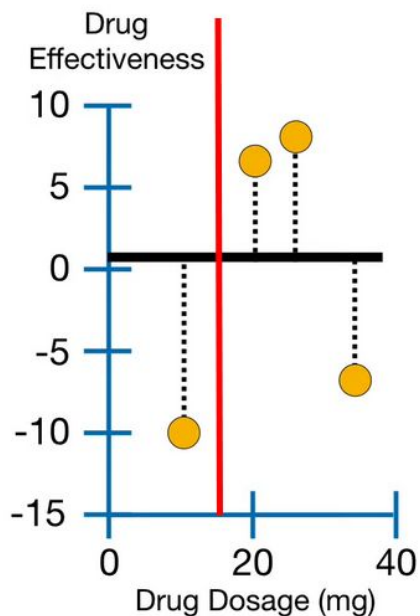
-10.5, 6.5, 7.5, -7.5

4

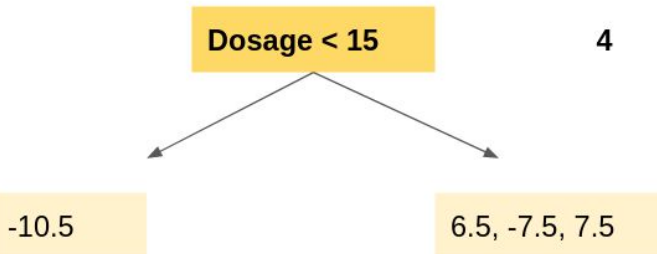
¿Podríamos agrupar estos
residuales de manera de
hacer una mejor predicción?

Vamos a centrarnos en los 2 primeros casos

Similarity Score



--->



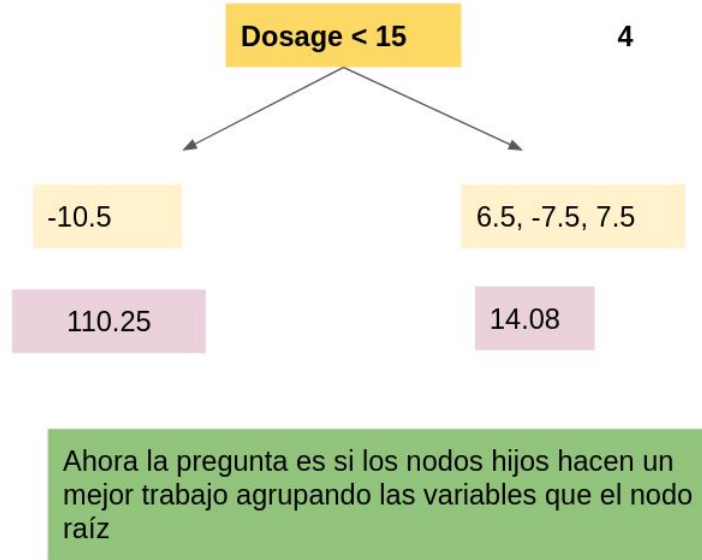
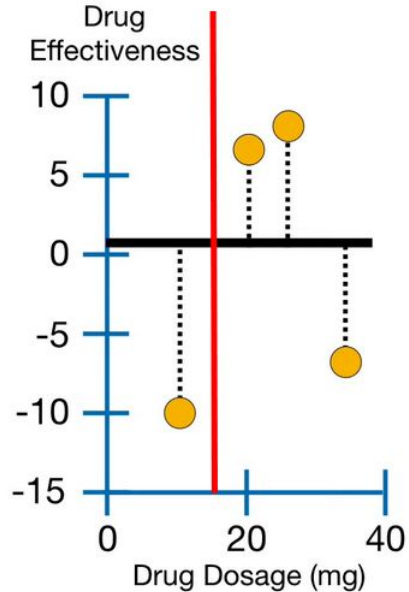
Vamos a calcular el Similarity Score para los residuales en las distintas ramas:

$$\text{Similarity Score} = \frac{-10.5^2}{1 + 0} = 110.25$$

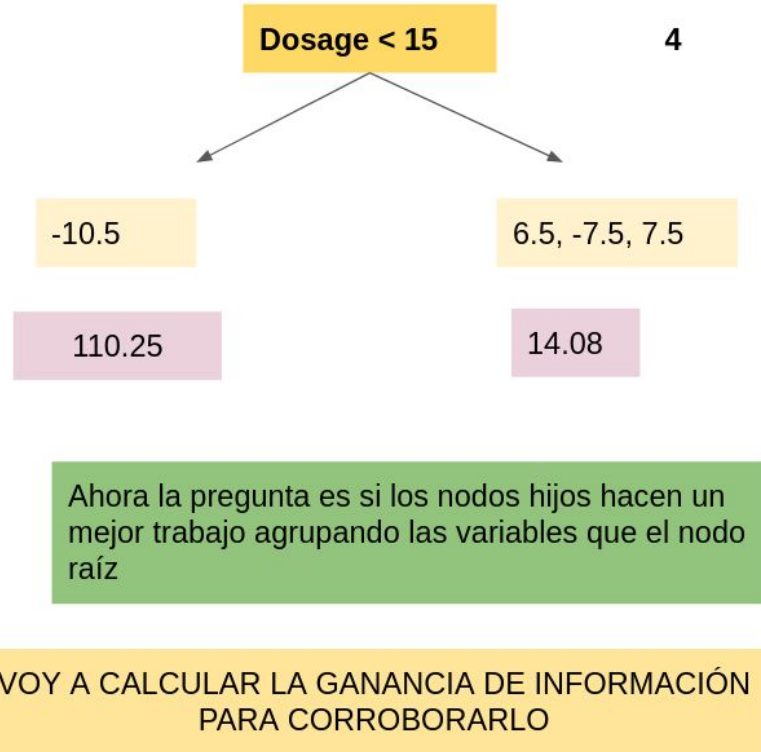
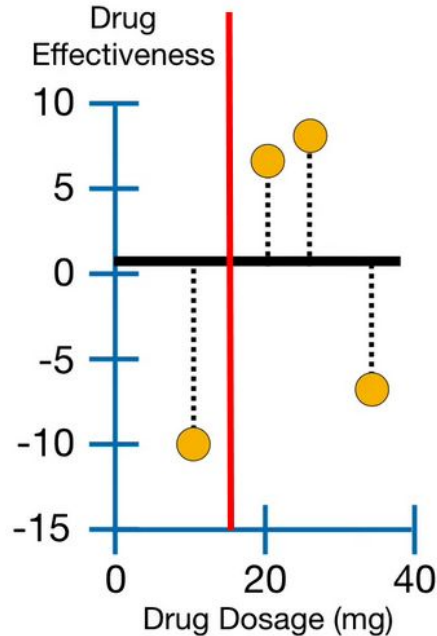
$$\text{Similarity Score} = \frac{(6.5 + 7.5 + -7.5)^2}{3 + 0}$$

$$\text{Similarity Score} = \frac{6.5^2}{3 + 0} = 14.08$$

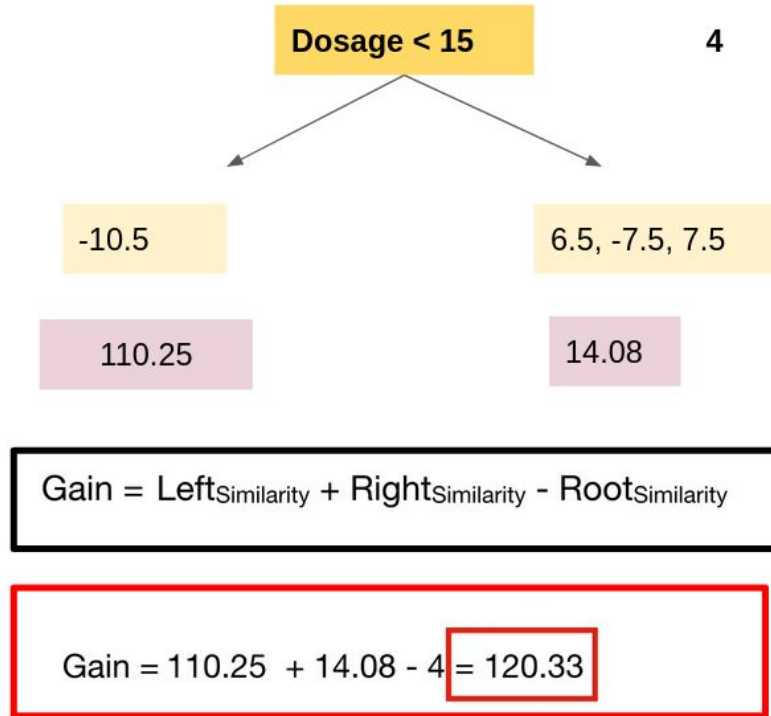
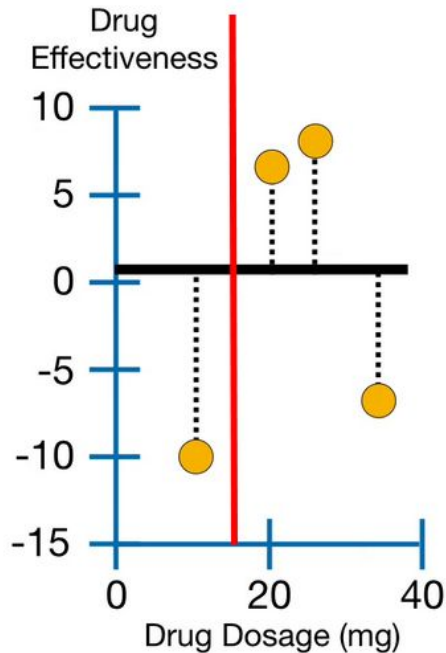
Similarity Score



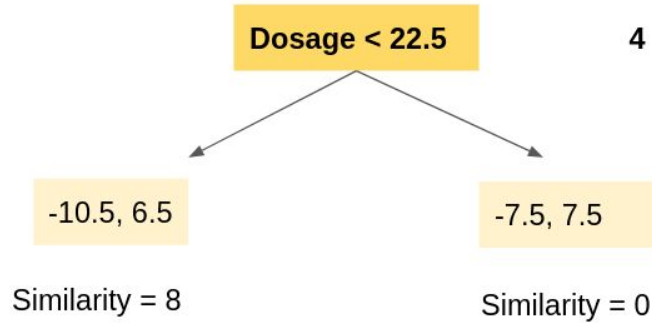
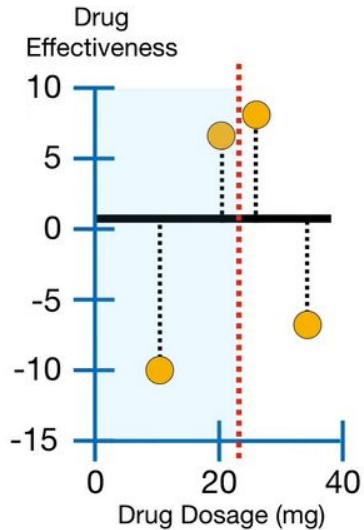
Similarity Score



Calculamos el Information Gain



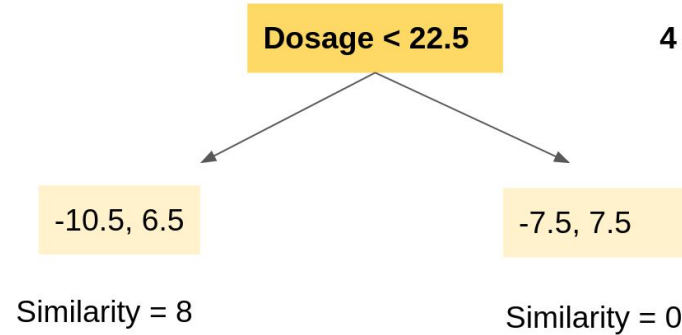
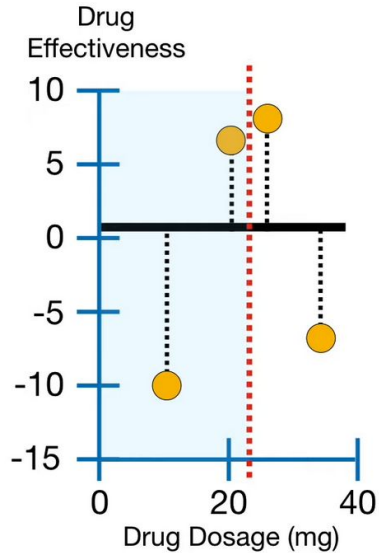
Threshold < 22.5



$$\text{Gain} = \text{Left}_{\text{Similarity}} + \text{Right}_{\text{Similarity}} - \text{Root}_{\text{Similarity}}$$

$$\text{Gain} = 8 + 0 - 4 = 4$$

Threshold < 22.5

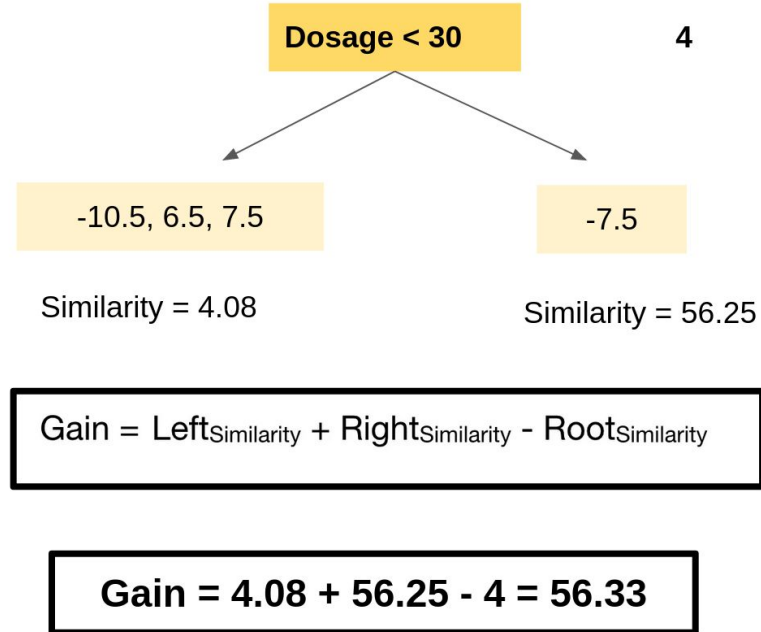
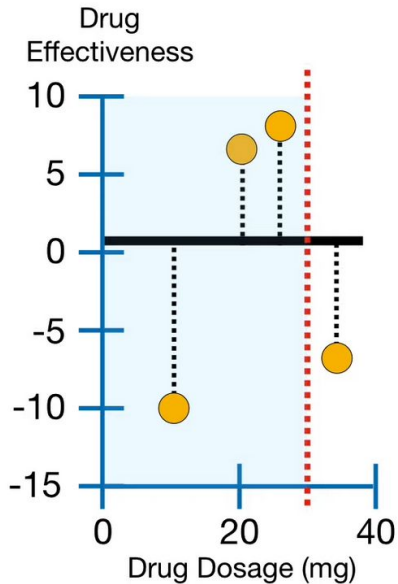


$$\text{Gain} = \text{Left}_{\text{Similarity}} + \text{Right}_{\text{Similarity}} - \text{Root}_{\text{Similarity}}$$

$$\text{Gain} = 8 + 0 - 4 = 4$$

La ganancia de información es menor que con el threshold < 15

Threshold < 30



La ganancia de información sigue siendo menor que con el threshold < 15

Nodo raíz

$$\text{Gain} = \text{LeftSimilarity} + \text{RightsSimilarity} - \text{RootSimilarity}$$

$$\text{Gain} = 110.25 + 14.08 - 4 = 120.33$$

$$\text{Gain} = \text{LeftSimilarity} + \text{RightsSimilarity} - \text{RootSimilarity}$$

$$\text{Gain} = 8 + 0 - 4 = 4$$

$$\text{Gain} = \text{LeftSimilarity} + \text{RightsSimilarity} - \text{RootSimilarity}$$

$$\text{Gain} = 4.08 + 56.25 - 4 = 56.33$$

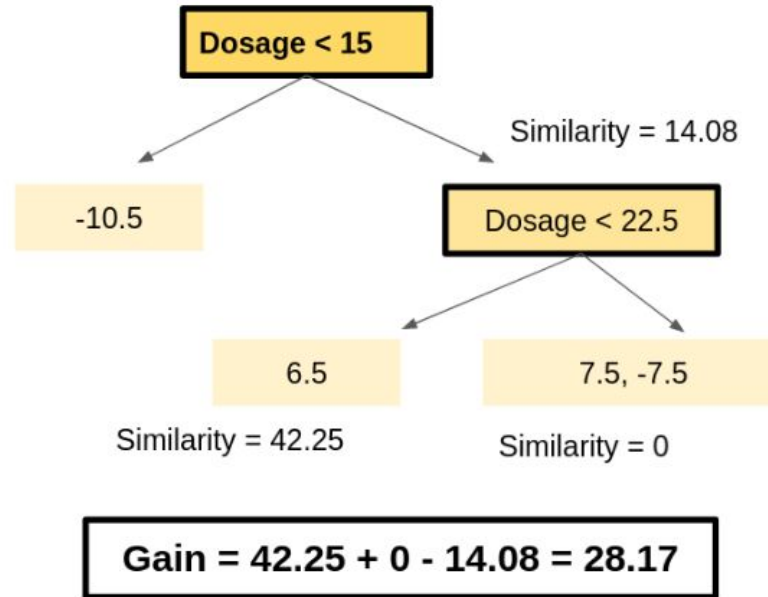
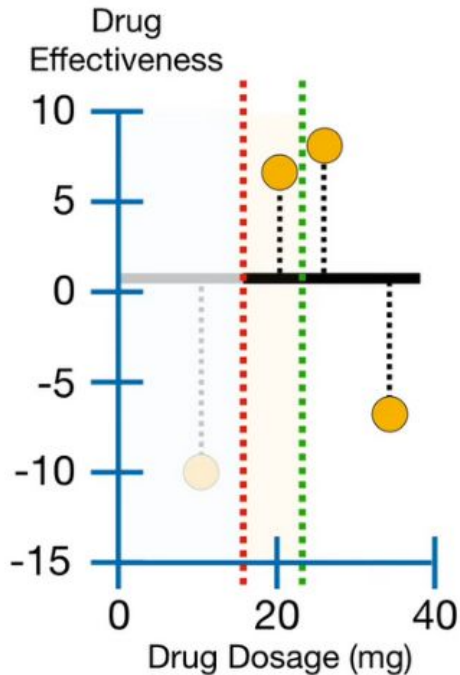
Dosage < 15



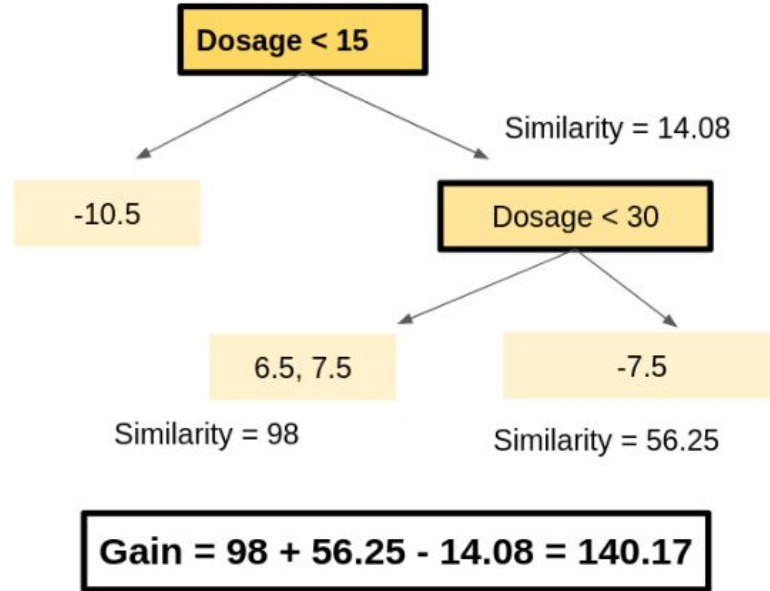
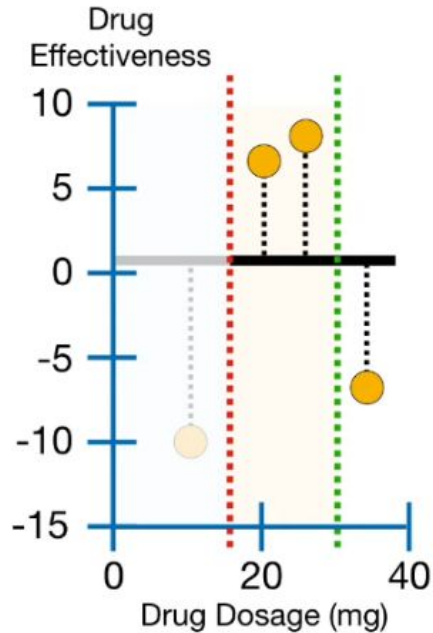
Dosage < 22.5

Dosage < 30

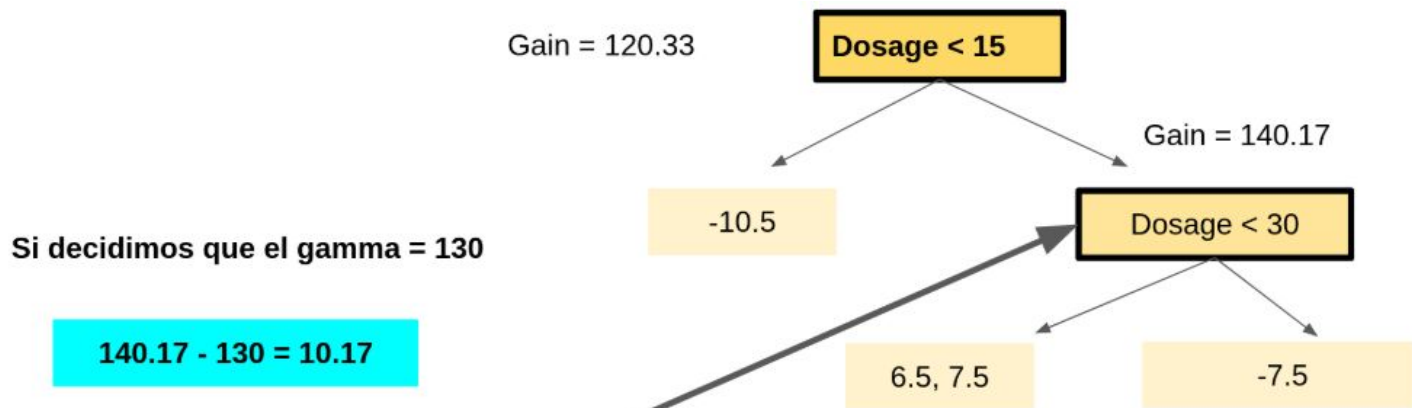
Vamos con la 2do rama



Vamos con la 2da rama



Cómo actúa el gamma



Si decidimos que el gamma = 130

$$140.17 - 130 = 10.17$$

Tenemos un valor positivo, por lo que no vamos a remover esta rama

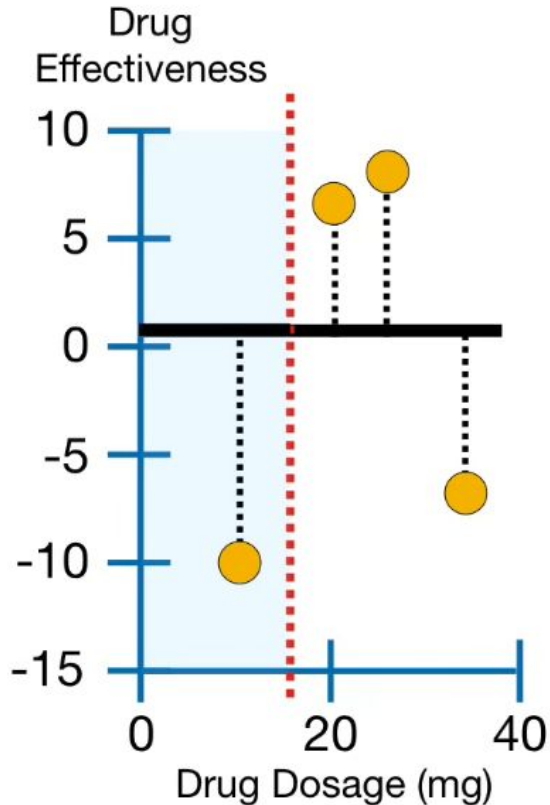
$$120.33 - 130 = -10.33$$

Este valor si bien es negativo, al tener una rama positiva, no removeremos el nodo raíz

Sin embargo, si decidimos que el gamma = 150

En ambos casos, la diferencia es negativa, y terminamos removiendo todo el árbol

Regularización

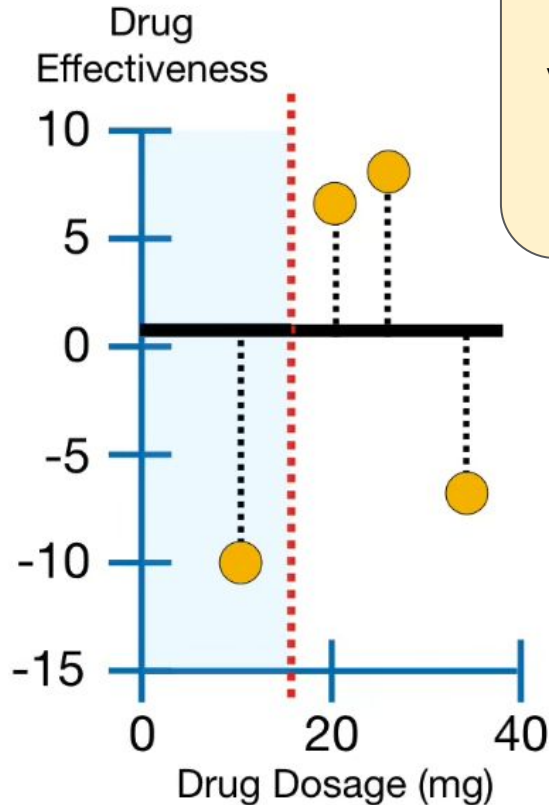


$$\text{Similarity Score} = \frac{\text{Sum of Residuals, Squared}}{\text{Number of Residuals} + \lambda}$$

...only this time, when we calculate **Similarity Scores**, we will set **λ (lambda) = 1**.

Recordar que λ (lambda) es un parámetro de regularización, lo que implica que pretende reducir el overfitting o la sensibilidad a observaciones individuales

Regularización



Vamos a calcular de vuelta el Similarity Score con el nuevo parámetro de regularización

-10.5, 6.5, 7.5, -7.5 Similarity = 3.2

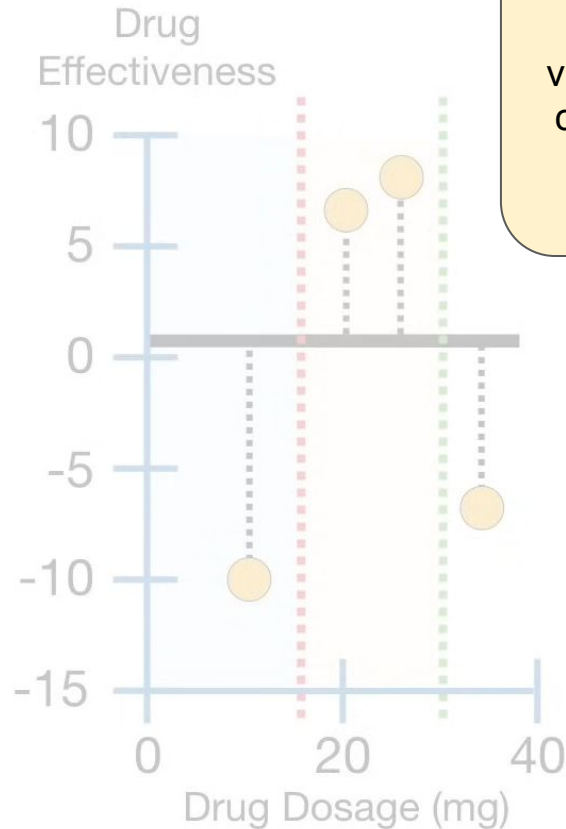
-10.5

6.5, 7.5, -7.5

$$\text{Similarity Score} = \frac{(-10.5 + 6.5 + 7.5 + -7.5)^2}{4 + \mathbf{1}} = \mathbf{3.2}$$

...**3.2**, which is **8/10s** of what we got when $\lambda = 0$.

Regularización



Vamos a calcular de vuelta el Similarity Score con el nuevo parámetro de regularización

Dosage < 15 Similarity = 3.2

-10.5

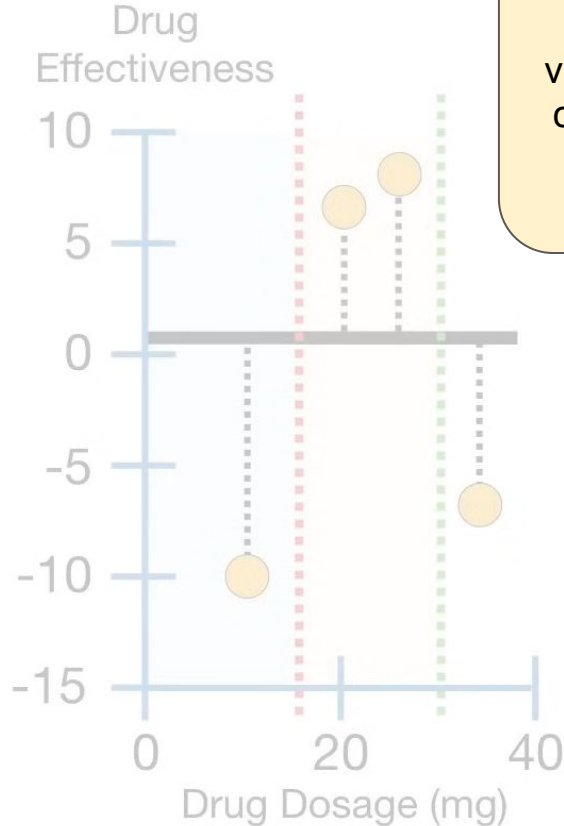
Similarity = 55.12

6.5, 7.5, -7.5

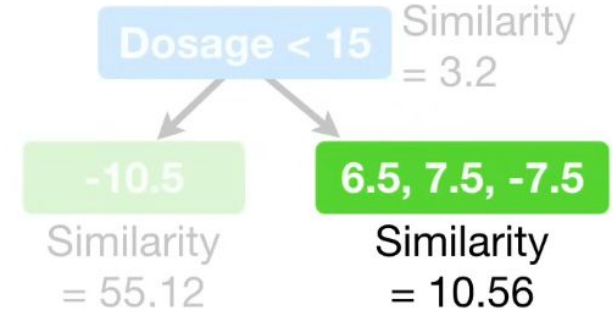
$$\text{Similarity Score} = \frac{-10.5^2}{1 + 1} = 55.12$$

...we get **55.12**, which is half of what we got when $\lambda = 0$.

Regularización



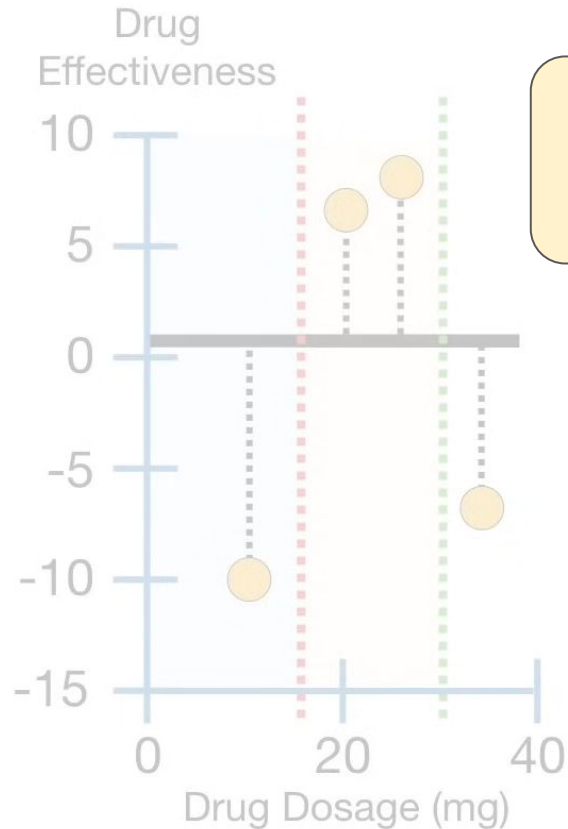
Vamos a calcular de vuelta el Similarity Score con el nuevo parámetro de regularización



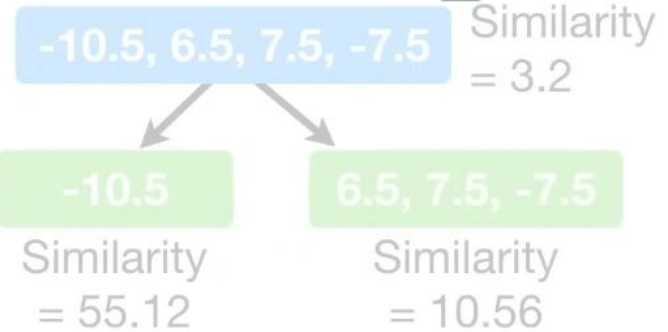
$$\text{Similarity Score} = \frac{(6.5 + 7.5 + -7.5)^2}{3 + 1} = 10.56$$

...we get **10.56**, which is **3/4s** of what we got when **$\lambda = 0$** .

Regularización



Nuevo cálculo del
Information Gain



$$\text{Gain} = 55.12 + 10.56 - 3.2 = 62.48$$

...we get **66**, which is a lot less than **120.33**, the value we got when $\lambda = 0$.

Regularización y γ (gamma)

Entonces, cuando $\lambda > 0$, es más fácil cortar hojas porque los valores de la ganancia de información (Gain) son más chicos.

When $\lambda = 1$...

Gain = 62.49

Dosage < 15

-10.5

Dosage < 30

Gain = 82.9

6.5, 7.5

-7.5

When $\lambda = 0$...

Gain = 120.33

Dosage < 15

-10.5

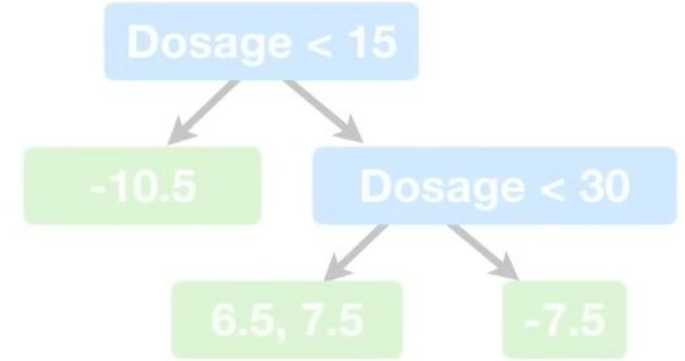
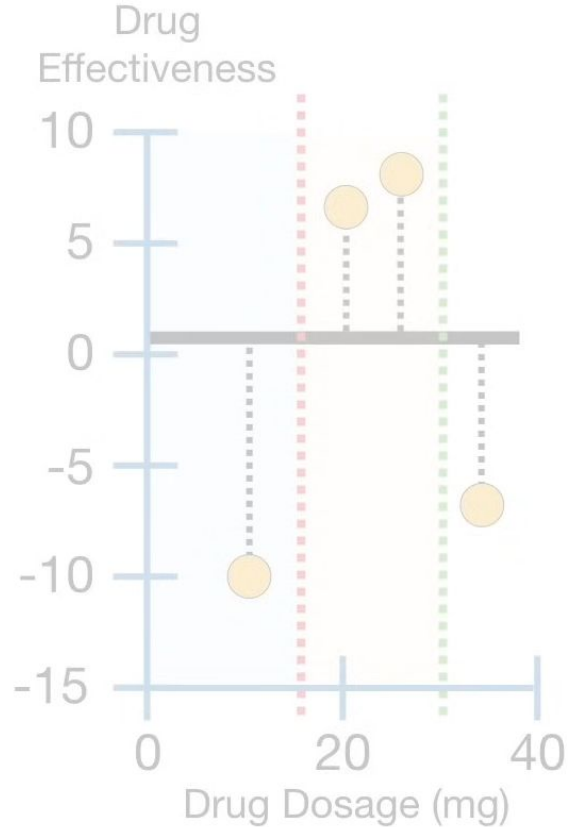
Dosage < 30

Gain = 140.17

6.5, 7.5

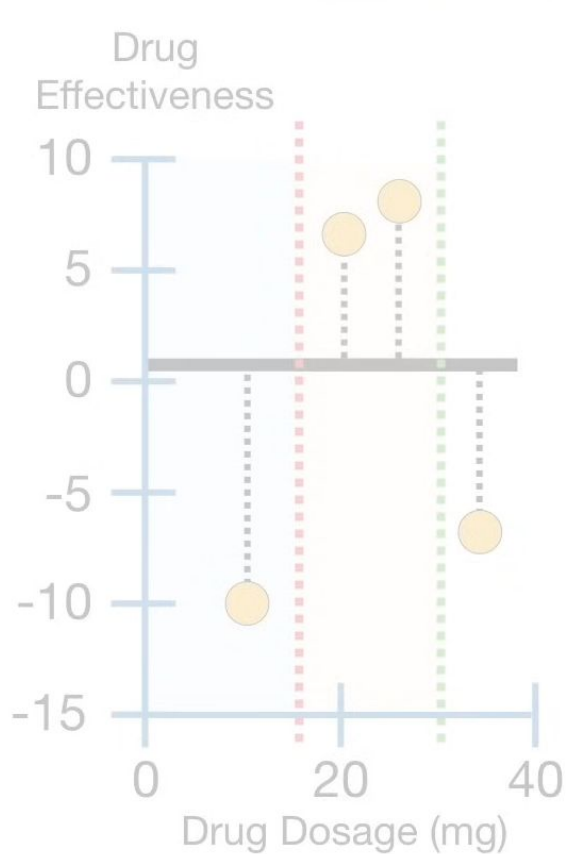
-7.5

Cálculo de la predicción



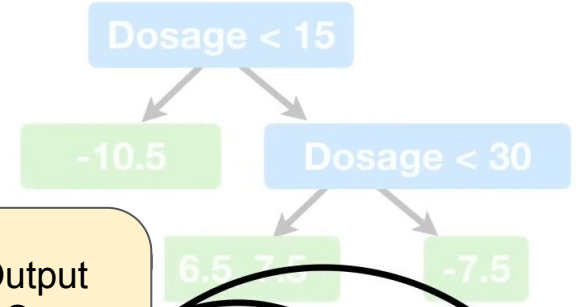
$$\text{Output Value} = \frac{\text{Sum of Residuals}}{\text{Number of Residuals} + \lambda}$$

Cálculo de la predicción



Residuals

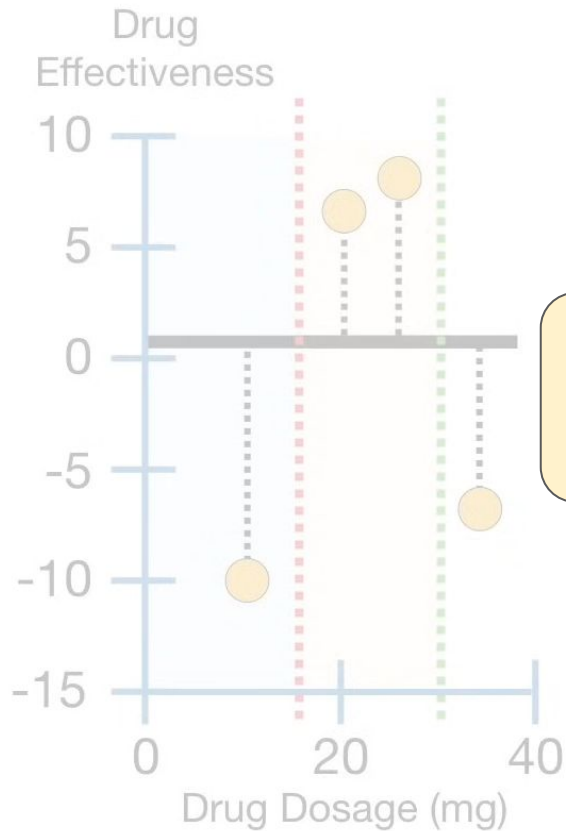
NOTA: el valor de salida (Output Value) es como el Similarity Score excepto que la suma de residuales NO está al cuadrado



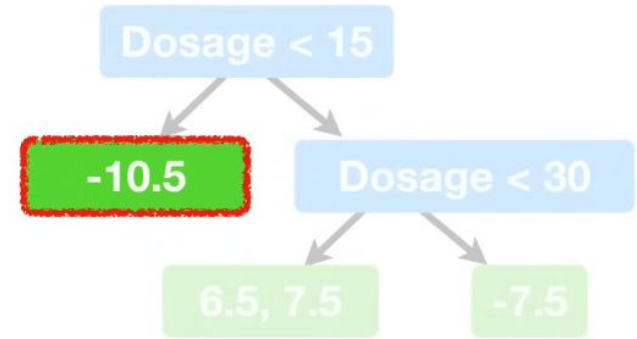
$$\text{Output Value} = \frac{\text{Sum of Residuals}}{\text{Number of Residuals} + 1}$$

$$\text{Similarity Score} = \frac{\text{Sum of Residuals, Squared}}{\text{Number of Residuals} + 1}$$

Cálculo de la predicción

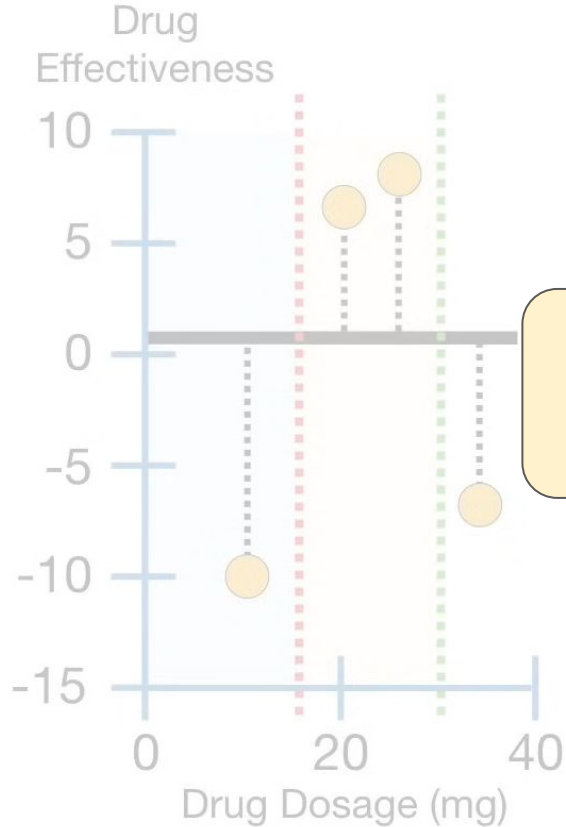


Si $\lambda = 0$, entonces **no hay regularización**, entonces el valor de salida es 10.5



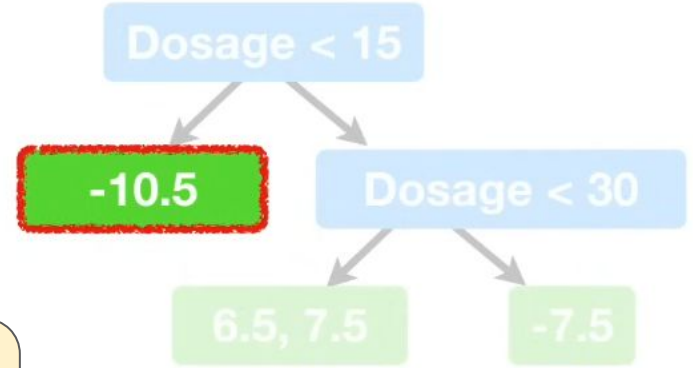
$$\text{Output Value} = \frac{-10.5}{1 + \mathbf{0}} = -10.5$$

Cálculo de la predicción



Si $\lambda = 0$, entonces **no hay regularización**, entonces el valor de salida es 10.5

$$\text{Output Value} = \frac{-10.5}{1 + 0}$$

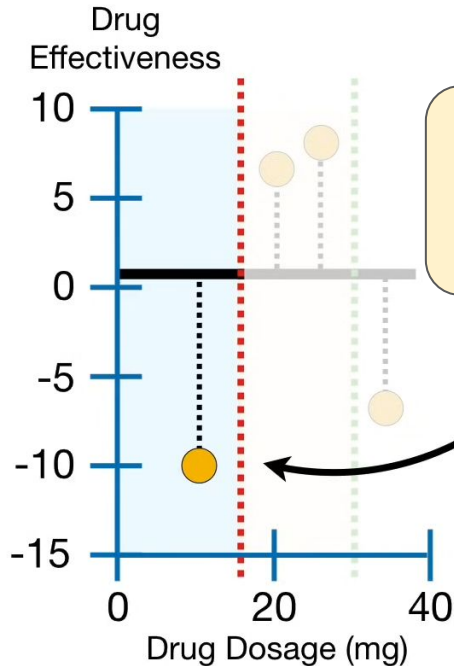


Cálculo de la predicción



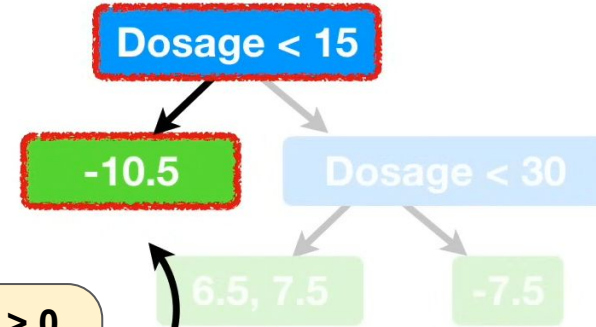
Predicted Drug Effectiveness

0.5



En otras palabras, cuando $\lambda > 0$, va a reducir lo que contribuye esta observación a la predicción general.

$$\text{Output Value} = \frac{-10.5}{1 + 1} = -5.25$$

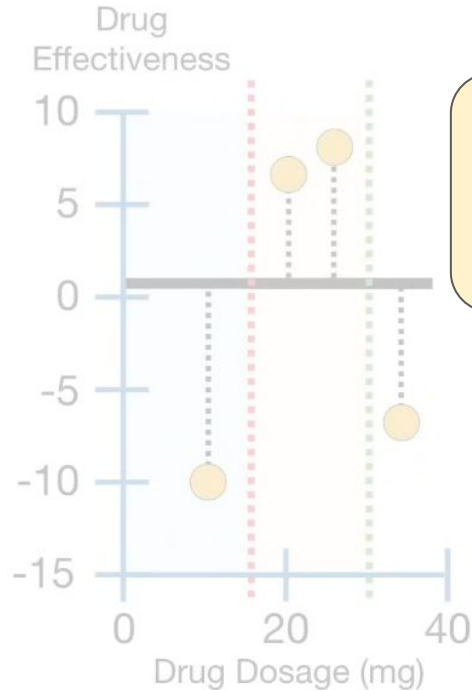


Cálculo de la predicción

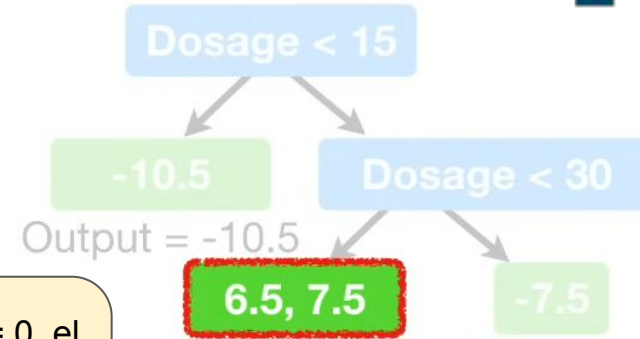


Predicted Drug Effectiveness

0.5

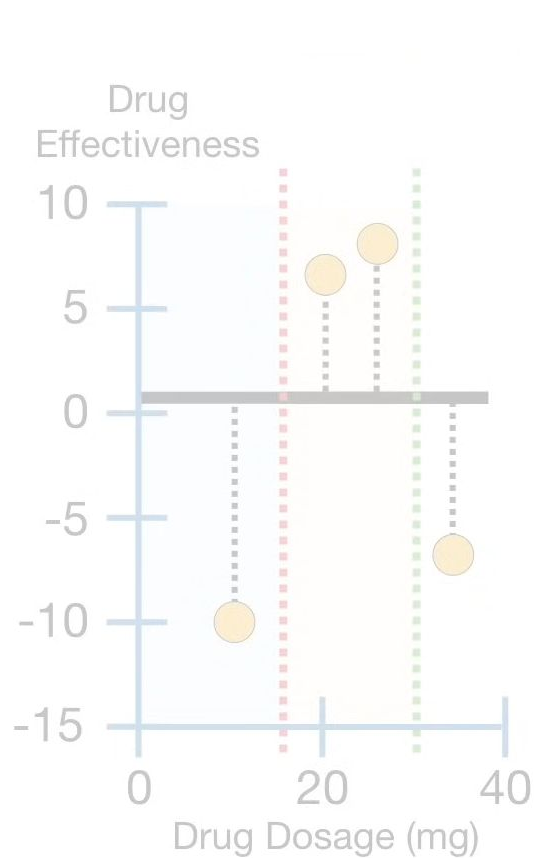


En otras palabras, cuando $\lambda = 0$, el valor de salida (Output value) para una hoja es simplemente la media de los residuos de la hoja.



$$\text{Output Value} = \frac{6.5 + 7.5}{2 + 0} = 7$$

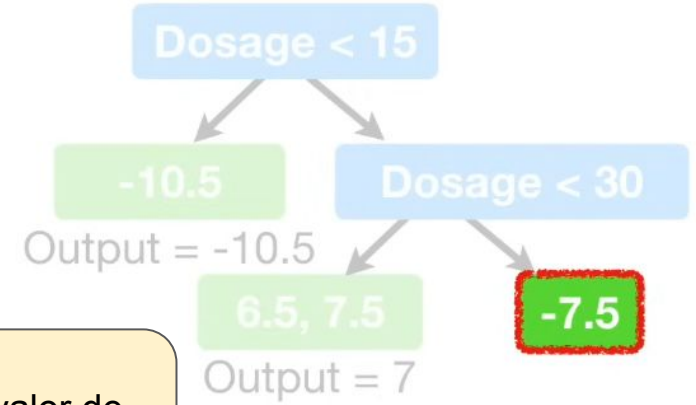
Cálculo de la predicción



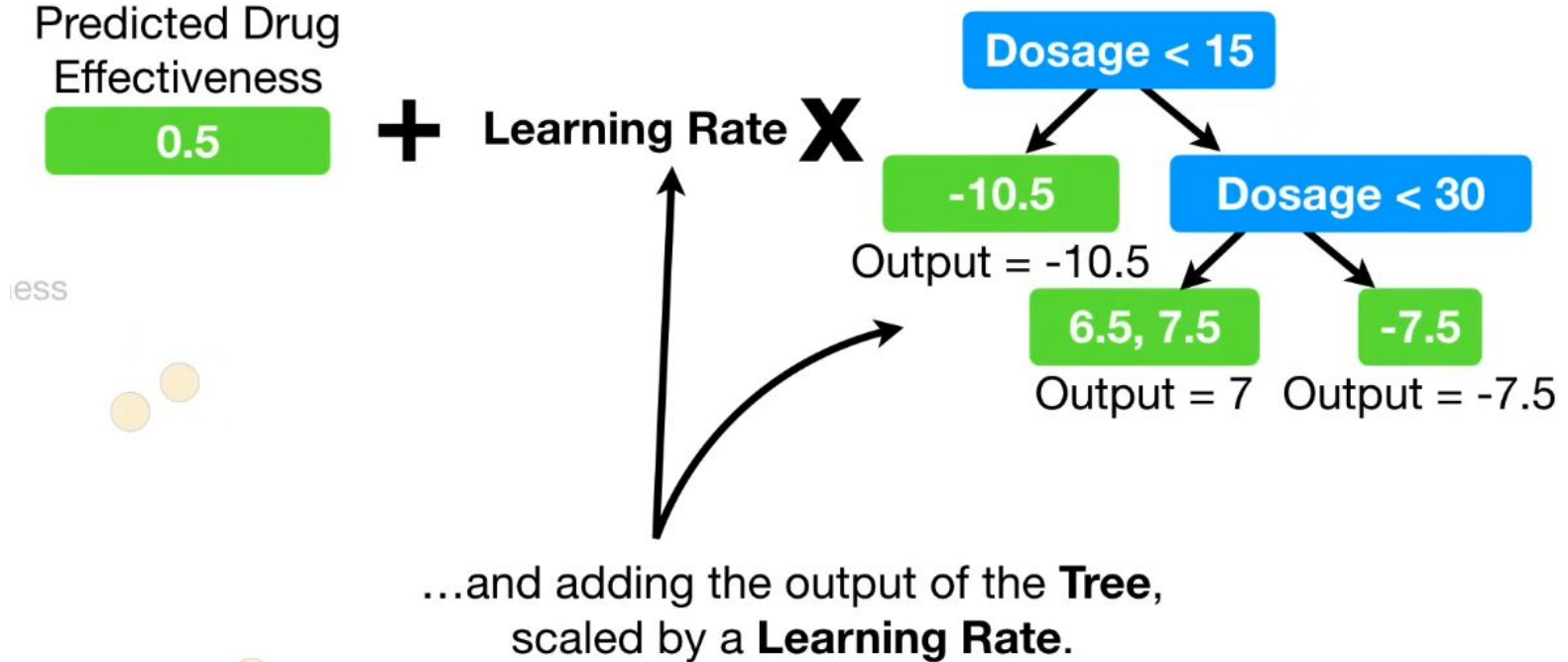
Drug
SS

Finalmente, cuando $\lambda = 0$, el valor de salida para esta hoja es:

$$\text{Output Value} = \frac{-7.5}{1 + 0}$$



Cálculo de la predicción



Cálculo de la predicción



Predicted Drug Effectiveness

0.5

+

0.3 X

Dosage < 15

-10.5

Output = -10.5

Dosage < 30

6.5, 7.5

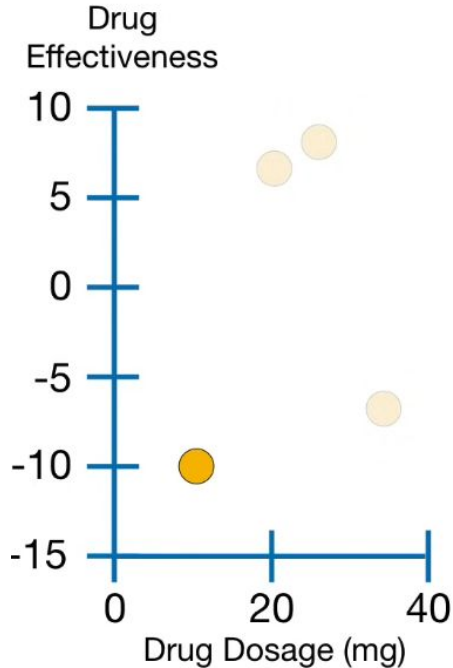
Output = 7

-7.5

Output = -7.5

$$0.5 + (0.3 \times -10.5) = -2.65$$

...and that gives us **-2.65**.



Cálculo de la predicción



Predicted Drug Effectiveness

0.5

+

0.3 X

Dosage < 15

-10.5

Output = -10.5

Dosage < 30

6.5, 7.5

Output = 7

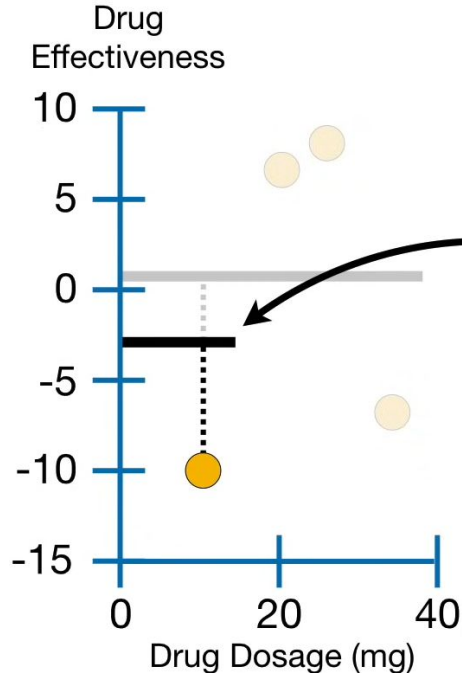
-7.5

Output = -7.5

$$0.5 + (0.3 \times -10.5) = -2.65$$

La nueva predicción es -2.65

El residual ha disminuido, lo que mejora nuestra predicción



Cálculo de la predicción



Predicted Drug Effectiveness

0.5

+

0.3 X

Dosage < 15

-10.5

Output = -10.5

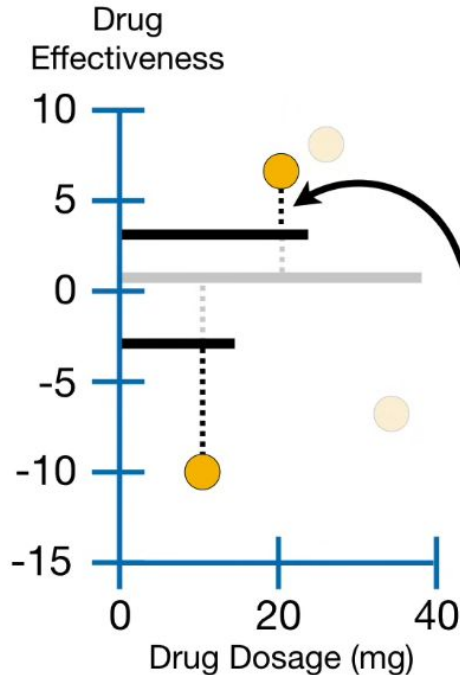
Dosage < 30

6.5, 7.5

Output = 7

-7.5

Output = -7.5



...and the new **Residual** is smaller than before, so we've taken another small step in the right direction.

- **Hiperparámetros relacionados con la arquitectura del modelo**

Estos hiperparámetros controlan cómo se construyen los árboles en el ensemble:

- **n_estimators**: Número total de árboles en el modelo.
- **learning_rate**: Tasa de aprendizaje que ajusta la contribución de cada árbol.
- **max_depth**: Profundidad máxima de los árboles.
- **min_child_weight**: Peso mínimo requerido en un nodo hoja para que pueda dividirse.
- **gamma** (o **min_split_loss**): Reducción mínima de pérdida necesaria para permitir una división.

- **Hiperparámetros relacionados con el muestreo**

Estos hiperparámetros introducen aleatoriedad en el entrenamiento para prevenir el sobreajuste:

- **subsample**: Fracción de filas del conjunto de datos que se utilizan en cada árbol.
- **colsample_bytree**: Porcentaje de características (columnas) seleccionadas aleatoriamente para cada árbol.
- **colsample_bylevel**: Porcentaje de características seleccionadas para cada nivel del árbol.
- **colsample_bynode**: Porcentaje de características seleccionadas para cada nodo.

- **Hiperparámetros relacionados con la regularización**

Estos ayudan a controlar el sobreajuste penalizando la complejidad del modelo:

- **lambda**: Regularización L2 (Ridge) aplicada a los coeficientes del modelo.
- **alpha**: Regularización L1 (Lasso) aplicada a los coeficientes del modelo.

- **Hiperparámetros relacionados con datos desbalanceados**

Controlan el comportamiento del modelo en conjuntos de datos con clases desbalanceadas:

- **scale_pos_weight**: Peso relativo entre las clases positivas y negativas (útil en problemas de clasificación binaria).
- **max_delta_step**: Restringe el tamaño de las actualizaciones en problemas desbalanceados.

Otras variantes de boosting

LightGBM fue desarrollado por Microsoft y está optimizado específicamente para **grandes conjuntos de datos**, priorizando la **velocidad** y la **eficiencia en memoria**.

Crecimiento de árboles: Utiliza un crecimiento de árboles **hoja-por-hoja** (*leaf-wise*), donde divide la hoja que promete la mayor reducción de pérdida, lo que a menudo resulta en árboles más profundos y asimétricos. Esto es **más rápido** que el enfoque nivel-por-nivel de XGBoost.

Optimización de datos (GOSS): Utiliza **Gradient-based One-Side Sampling (GOSS)** para excluir instancias con gradientes pequeños (errores bajos) y enfocarse solo en las instancias con **gradientes grandes** (errores grandes), acelerando el entrenamiento.

Optimización de características: Emplea **Exclusive Feature Bundling (EFB)** para agrupar características mutuamente exclusivas (que rara vez son distintas de cero simultáneamente) en un solo *bundle*, lo que reduce la dimensionalidad.

CatBoost fue desarrollado por Yandex (el "Google ruso") y se destaca principalmente por su capacidad de manejar **variables categóricas** de forma nativa y eficiente.

Manejo de categorías: **Codifica automáticamente** las variables categóricas de forma avanzada (usando estadísticas sobre el *target*) y durante el entrenamiento, sin necesidad de preprocesamiento (como *One-Hot Encoding*).

Ordered Boosting: Utiliza un esquema especial de *boosting* ordenado (*ordered boosting*) para evitar el ***target leakage*** (filtración del objetivo) que ocurre a menudo cuando se utilizan las estadísticas del *target* para codificar categorías. Esto mejora la generalización.

Árboles simétricos: Construye **árboles simétricos** (balanceados), lo que permite una inferencia más rápida en producción, aunque puede ser menos flexible que los árboles asimétricos de LightGBM/XGBoost.

Robusto al sobreajuste: Generalmente, es muy **robusto** y requiere menos ajuste de hiperparámetros que otros, gracias a su mecanismo de *Ordered Boosting* y a su manejo interno de la codificación.

XGBoost, LightGBM y CatBoost

Algoritmo	Innovación clave	Principal ventaja
XGBoost	Regularización (L1/L2) y Paralelización	Rápido y muy robusto, minimiza la función de pérdida con regularización
LightGBM	Leaf-wise tree growth (crecimiento por hoja)	Extremadamente rápido en grandes datasets, menor uso de memoria.
CatBoost	Manejo nativo de variables categóricas	Excelente rendimiento sin preprocesamiento complejo de categorías.

Documentación oficial



- Documentación oficial de XGBoost:

https://xgboost.readthedocs.io/en/release_3.2.0/index.html



- Documentación de Catboost:

<https://catboost.ai/>



CatBoost

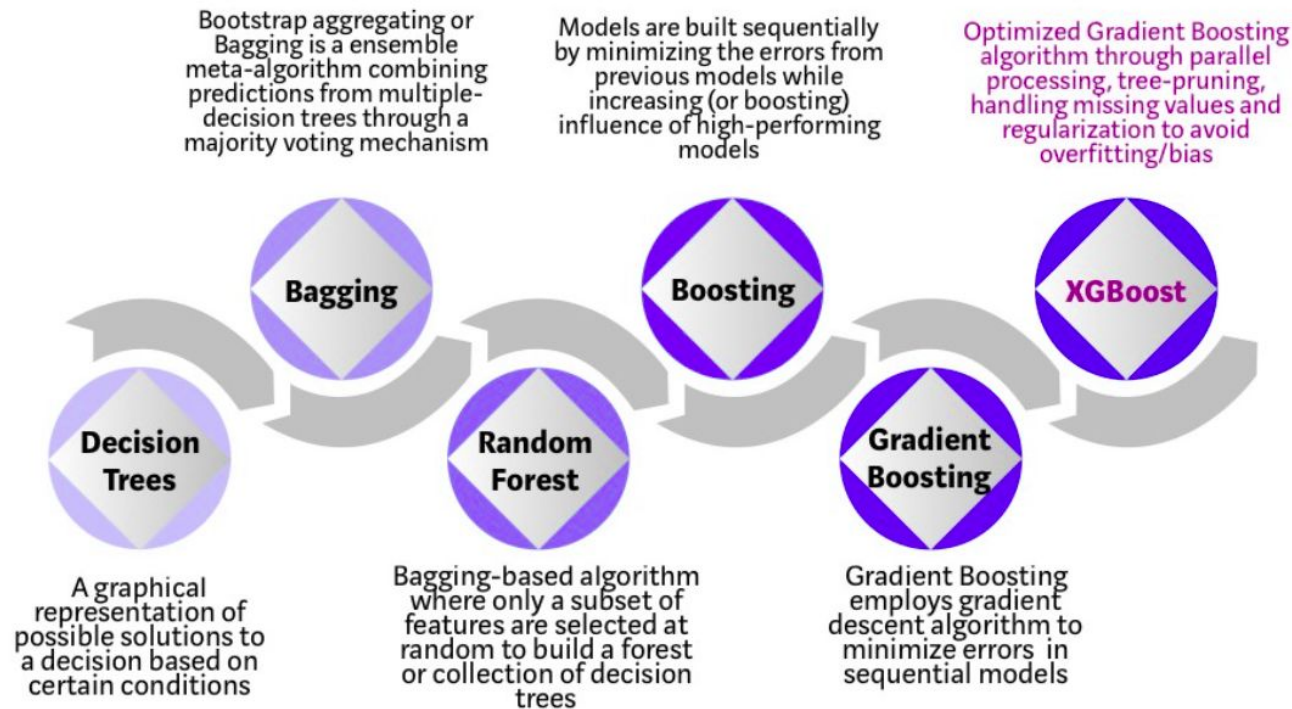
- Documentación de LightGBM:

<https://lightgbm.readthedocs.io/en/stable/>



LightGBM

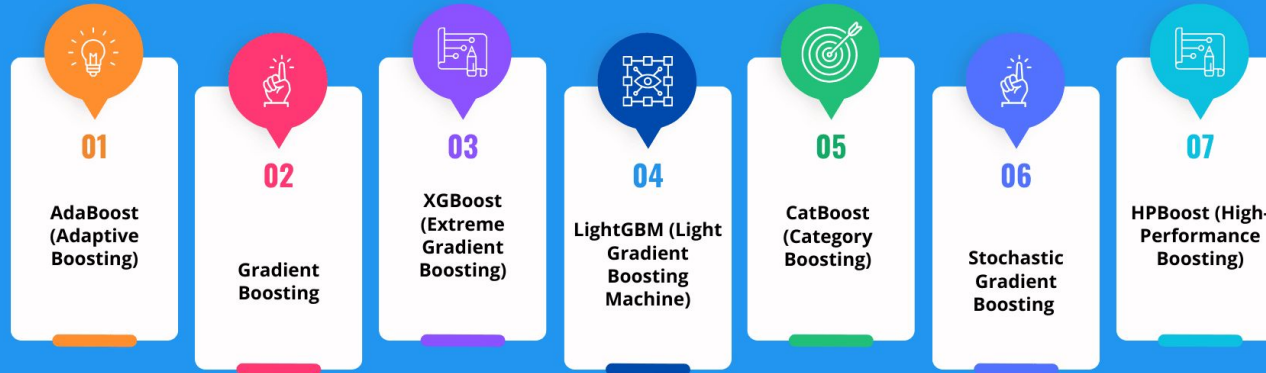
De árboles de decisión a XGBoost



Fuente:

<https://towardsdatascience.com/https-medium-com-vishalmorde-xgboost-algorithm-long-she-may-rein-edd9f99be63>

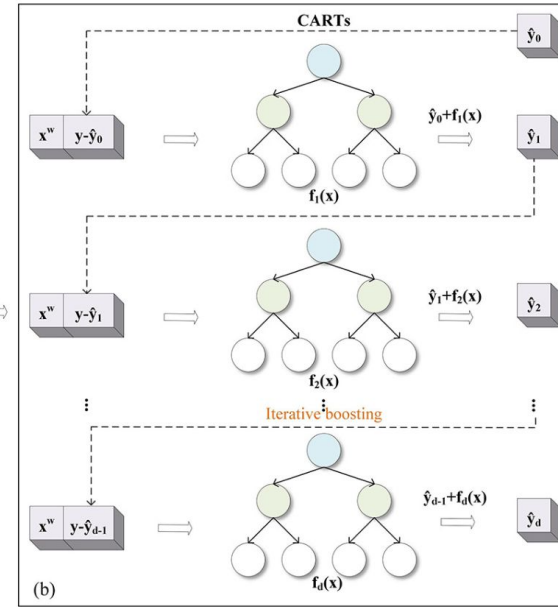
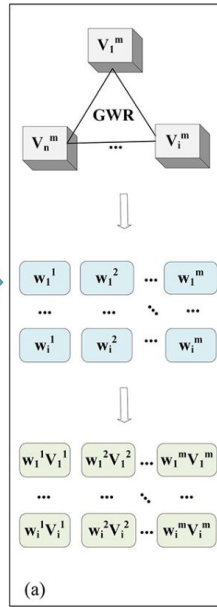
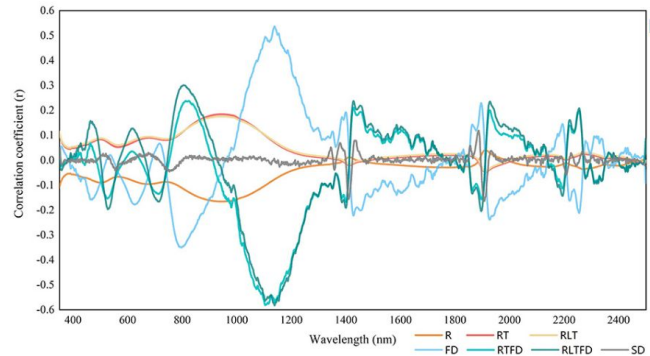
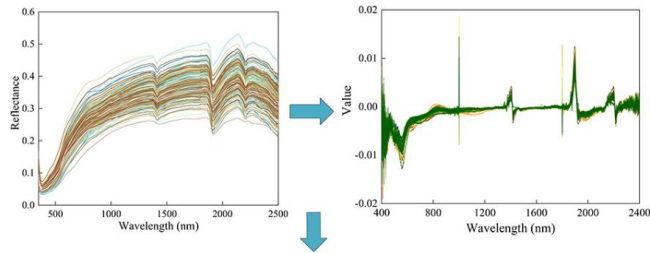
Boosting Algorithms



dataaspirant.com

Aplicaciones de Boosting

Estimation of the soil arsenic concentration using a geographically weighted XGBoost model based on hyperspectral data



Prediction of Bike-sharing Trip Counts: Comparing Parametric Spatial Regression Models to a Geographically Weighted XGBoost Algorithm

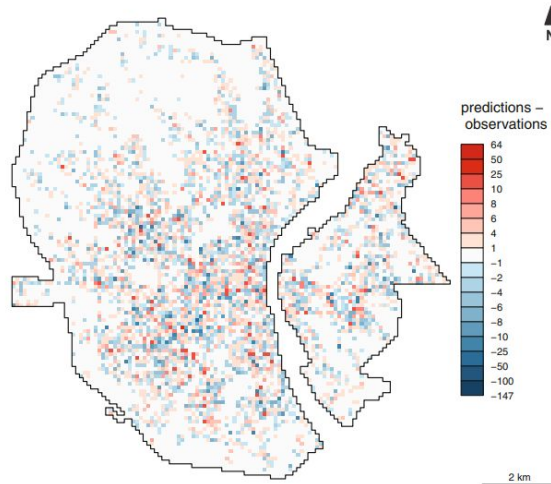


Figure 12. Difference between predicted and observed values of the test data set, predictions determined using XGBoost.

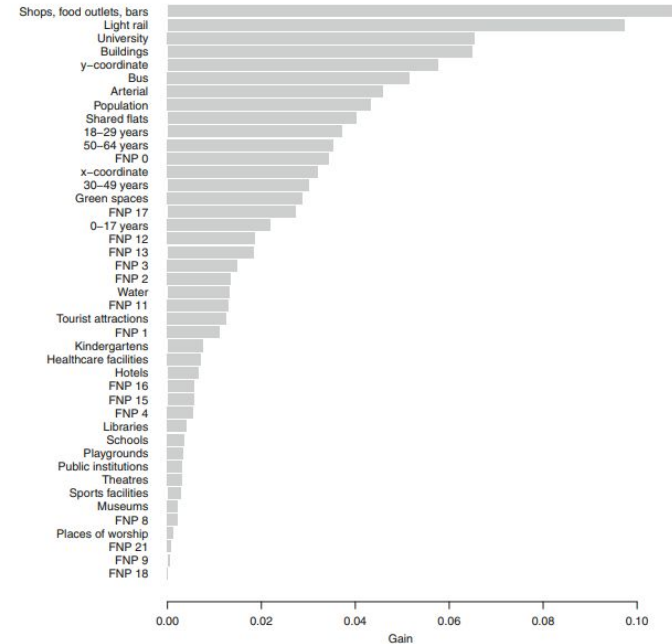
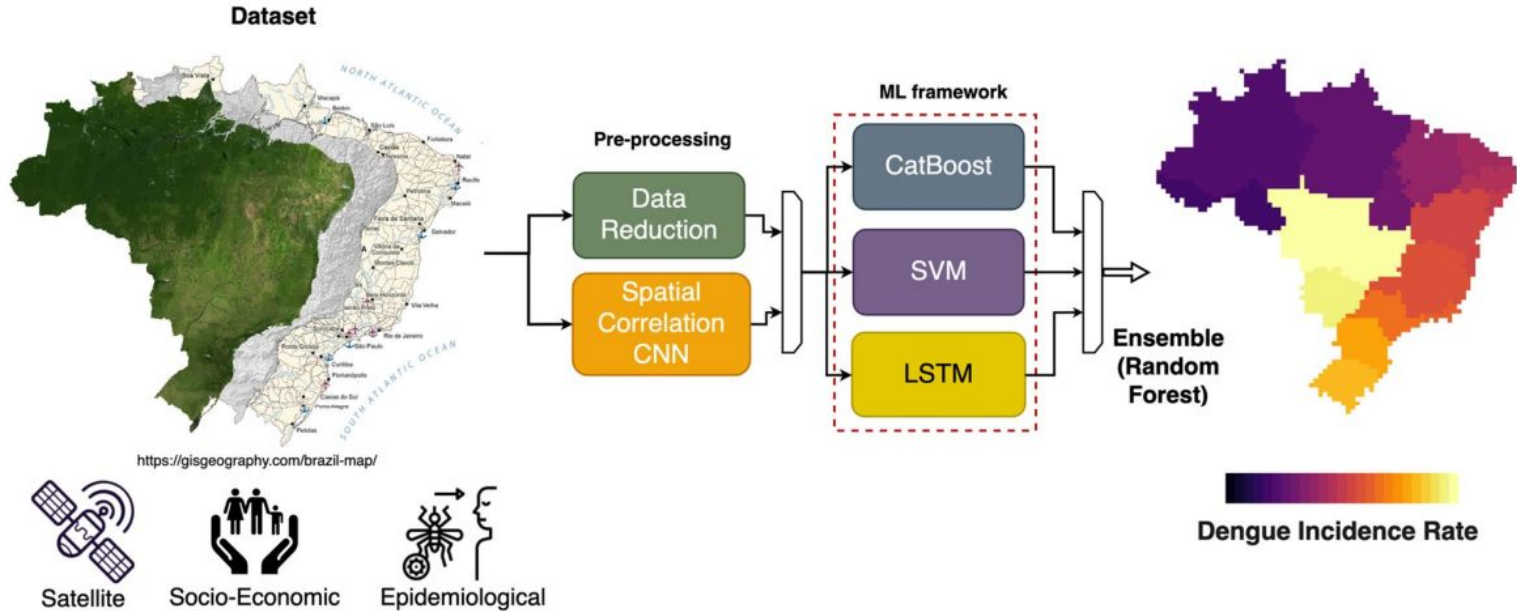
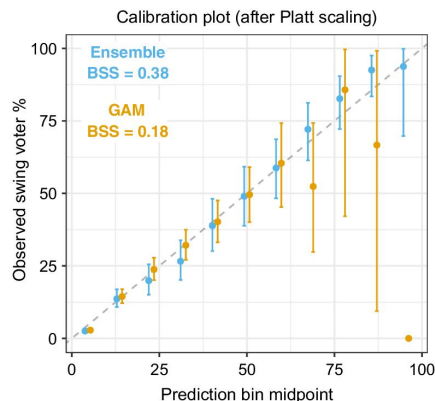
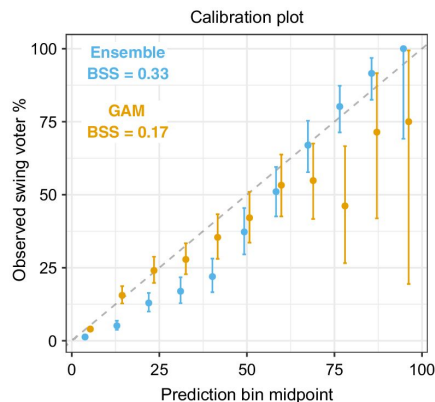
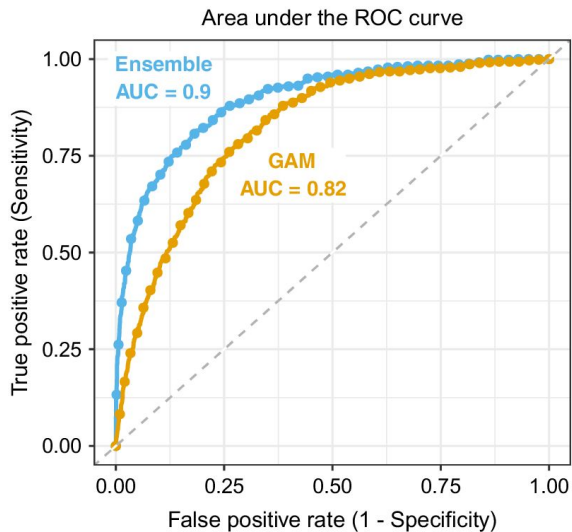


Figure 6. Importance plot of the features in the XGBoost model – gain.

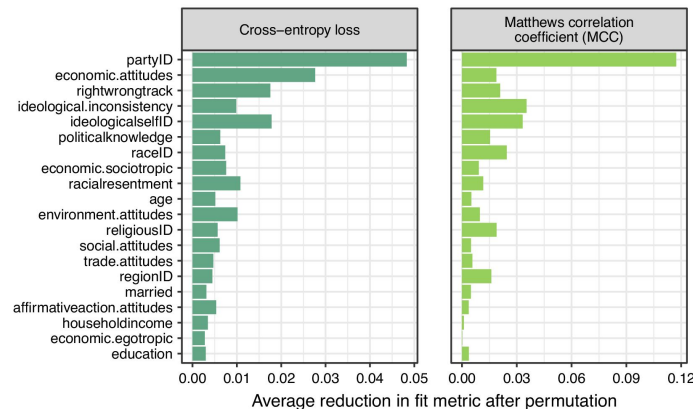
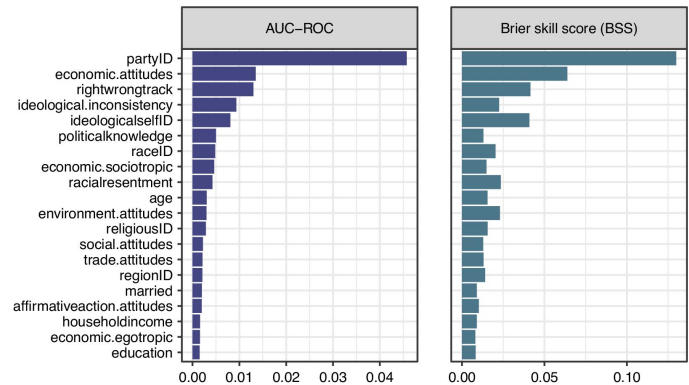
A responsible ensemble machine learning approach to forecast dengue outbreaks



Measuring swing voters with a supervised machine learning ensemble



All voters Unconditional permutation



Recordar

Regresión lineal

Regresión

Regresión logística

Clasificación

SVM

Árboles de decisión

Random Forests

Boosting

Regresión y clasificación

Con SVM, DT, RF y Boosting (en diversas sus variantes) podemos hacer regresiones y clasificaciones

- Statquest (video sobre Adaboost) <https://www.youtube.com/watch?v=LsK-xG1cLYA>
- Statquest (video sobre Gradient Boosting):
<https://www.youtube.com/watch?v=3CC4N4z3GJc&t=518s>
- Statquest (video sobre XGBoost): <https://www.youtube.com/watch?v=OtD8wVaFm6E>
- <https://zia207.github.io/geospatial-r-github.io/exboost.html>
- <https://towardsdatascience.com/https-medium-com-vishalmorde-xgboost-algorithm-long-sh-e-may-rein-edd9f99be63d>
- Apuntes de clase de Sebastian Raschka (bagging, RF, boosting)
https://sebastianraschka.com/pdf/lecture-notes/stat451fs20/07-ensembles__notes.pdf
- <https://blog.dailydoseofds.com/p/top-gradient-boosting-methods>
- <https://blog.dailydoseofds.com/p/48-most-popular-open-ml-datasets>
- <https://www.riskified.com/resources/article/boosting-comparison/>
- Machine learning with R. Capítulo 14
- Sebastian Raschka. Gradient Boosting <https://www.youtube.com/watch?v=zblsrxc7XpM>
- <https://datos.ninja/tutorial/xgboost/>